

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/314975149>

A model-driven approach for predicting and analysing the execution efficiency of multi-core processing

Article in *International Journal of Computational Science and Engineering* · January 2017

DOI: 10.1504/IJCSE.2017.10003827

CITATIONS

3

READS

177

3 authors:



Khondker Shajadul Hasan

University of Houston - Clear Lake

46 PUBLICATIONS 211 CITATIONS

[SEE PROFILE](#)



John Antonio

University of Oklahoma

102 PUBLICATIONS 1,016 CITATIONS

[SEE PROFILE](#)



Sridhar Radhakrishnan

University of Oklahoma

163 PUBLICATIONS 1,742 CITATIONS

[SEE PROFILE](#)

A model-driven approach for predicting and analysing the execution efficiency of multi-core processing

Khondker S. Hasan*

College of Science and Engineering,
University of Houston-Clear Lake (UHCL),
Houston, TX 77058, USA
Email: HasanK@uhcl.edu
*Corresponding author

John K. Antonio

School of Computer Science,
University of Oklahoma,
210 W. Boyd Street,
Norman, OK 73019, USA
Email: antonio@ou.edu

Sridhar Radhakrishnan

School of Computer Science,
University of Oklahoma,
110 W. Boyd Street,
Norman, OK 73019, USA
Email: sridhar@ou.edu

Abstract: Techniques for predicting the efficiency of multi-core processing associated with a set of tasks with varied CPU and main memory requirements are introduced. Prediction of CPU and memory availability is important in the context of making process assignment, load balancing, and scheduling decisions in distributed systems. Given a set of tasks each with varied CPU and main memory requirements, and a multi-core system (which generally has fewer cores than the number of tasks), we provide upper- and lower-bound models (formulas) for the efficiency with which the tasks are executed. In addition, a model for average CPU availability is introduced from the empirical study for applications that require a single predicted value instead of bounds. To facilitate scientific and controlled empirical evaluation, real-world benchmark programs with dynamic behaviour (CPU and memory requirements change in a short interval of time) are employed on UNIX systems that are parameterised by their CPU usage factor and memory requirement.

Keywords: composite prediction model; CPU availability; concurrent threads; memory model; modelling and prediction; multi-core processing; performance evaluation.

Reference to this paper should be made as follows: Hasan, K.S., Antonio, J.K. and Radhakrishnan, S. (2017) 'A model-driven approach for predicting and analysing the execution efficiency of multi-core processing', *Int. J. Computational Science and Engineering*, Vol. 14, No. 2, pp.105–125.

Biographical notes: Khondker S. Hasan is currently working as an Assistant Professor of Computer Science at the University of Houston, Clear Lake, Houston, TX 77058, USA. He received his PhD in Computer Science from the University of Oklahoma, OK, USA in 2014. He received his MS in Computer Science from Wichita State University, Kansas, USA in 2001 and BS in Computer Science from North South University, Dhaka, Bangladesh in 1998. He has worked as a Lecturer and Senior Lecturer from 2002 to 2009 in the Computer Science Department, North South University. During his graduate studies, he has worked as a Research Associate at MSCI Inc. He has received the Robertson Research and Travel Grants in 2014 and also the recipient of the Computer Science Advisory Board Scholarship 2012–2013. His research interest includes parallel and distributed systems, performance modelling, and real-time scheduling.

John K. Antonio received his BS, MS, and PhD in Electrical Engineering from Texas A&M University, College Station. He is currently a Senior Associate Dean of Engineering and Professor of Computer Science at the University of Oklahoma (OU), Norman, OK 73019, USA. From 1999 to 2006, he served as the Director and a Professor of Computer Science with the University of Oklahoma (OU), Norman. From 2011 to 2013 he was an Executive Director with MSCI. Before joining OU, he was on the Faculty of Electrical and Computer Engineering with Purdue University, West Lafayette, IN, and he was also on the Faculty of Computer science with Texas Tech University, Lubbock. Numerous agencies and companies have supported his research over the years. He has been a Principal Investigator (PI) or Co-PI on more than 25 sponsored research projects totalling more than \$2.5 million. He has co-authored more than 100 publications and reports. He is the holder of a US patent granted in 2010.

Sridhar Radhakrishnan is a Professor and Director of the School of Computer Science at the University of Oklahoma, where he joined in 1990. He received his PhD in Computer Science from Louisiana State University in 1990. He received his undergraduate degrees from Vivekananda College, Chennai, India and from University of South Alabama, Mobile, Alabama. His research interest is in the design of protocols for broadband, wireless and mobile networks. He has published many research articles in journals, conference proceedings, and book chapters. His research is supported by industry and government.

This paper is a revised and expanded version of a paper entitled ‘A new composite CPU/memory model for predicting efficiency of multi-core processing’ presented at 5th Intl. Workshop on SoCs, Heterogeneous Architectures and Workloads (SHAW-5) held in conjunction with High Performance Computer Architecture (HPCA-20), Orlando, FL, 15–19 February 2014.

1 Introduction

The success of approaches for assigning threads (or processes) to multi-core systems relies on the existence of reasonably accurate models for estimating resource (CPU and memory) availability. System efficiency prediction involves estimating the system’s behaviour for the set of tasks to be executed on it. Making such predictions is complicated owing to the dynamic nature of the system and its workload, which can vary drastically in a short span of time (Beltrán et al., 2008). For any prediction approach, it can be useful to know *a priori* certain characteristics of tasks that are planned to be assigned to the system. As an example, it is useful to know the maximum amount of main memory a task will consume at any point during its execution (*memory requirement*). This information is useful to forecast the amount of time that may be consumed for memory paging activities. It is also useful to know the *CPU requirement* of the task, which is the fraction of time a task requires the CPU.

While *a priori* information such as memory and CPU requirements of tasks is useful, it is easier to obtain in some cases and more difficult in others. For example, in the case of Merge sort we can approximately determine the memory requirement based on the number of elements to be sorted. Tasks such as generating prime numbers, computing Fast Fourier Transforms, and related others require significant use of the CPU. One can also know the CPU and memory requirements of a task based on the information gathered from its earlier executions (Aioanei, 2011). The execution of many scientific models such as economic, meteorological, and geographical models fall into this category, where the program remains the same and the data on which it operates changes over time.

Given the CPU requirements of tasks in a run queue, Beltrán et al. (2008) provide an analytical model to estimate the *CPU availability* (which is the percentage of CPU time that will be allocated) for the new task prior to its placement in the run queue. It is shown that in certain cases this information can be used to schedule the execution of tasks in such a way that the completion time of all the tasks is minimised. The work of Beltrán et al. (2008) has been extended here as follows: for a batch of tasks (each with its own CPU requirements), an analytical model is introduced to determine the CPU availability using the *sum total* of the CPU requirement of each of the tasks in the batch. Using this sum total posed a challenge because the CPU availability prediction is precise only when the order of task execution is known *a priori*. To address this challenge, the analytical model introduced in this paper provides upper- and lower-bounds on CPU availability. These bounds are necessary since the actual CPU availability depends on the order of execution of tasks in the batch. Thus, the analytical model in Hasan et al. (2014) is oblivious to the CPU scheduler.

When a processor accesses memory, it spends a significant amount of time waiting for the data to become available because of cache misses which may result in up to 50% of stall time. This situation generates huge overhead when the frequency of memory access increases. To overcome this situation, most of the recent hardware designs have implemented multi-threaded processor cores in which two or more hardware threads are assigned to each core. That way, if one thread stalls while waiting for memory, the core can switch to another thread (Avi et al., 2010). To maintain its own architectural state, each core has its independent register set and thus appears to the operating system to be a separate physical processor. From an

operating system perspective, each hardware thread (when hyper-threading is enabled) appears as a logical processor that is available to run a software thread. Thus, on a dual-threaded, dual-core system, four logical processors are presented to the operating system. We have incorporated the effect of hyper-threading in our new CPU availability and memory models for accuracy.

Further improvements to Hasan et al. (2014) are presented in the present paper; these are summarised as follows:

- First, a *new CPU availability lower-bound model* is introduced which provides a significantly tighter bound compared with the previous lower-bound model.
- Second, a new *average CPU availability model* is introduced, which is derived from the empirical analysis of the CPU availability model. This model can predict the average CPU availability for a set of tasks before placing them into the run-queue. The model provides a single CPU availability prediction value (instead of upper- and lower-bounds) for the set of tasks without explicit knowledge of the mapping between available cores and tasks.
- Third, the CPU availability model (consisting of the new lower-bound) is combined non-trivially with the memory model to derive a *composite model*. This model consists of analytically derived upper- and lower-bounds for predicting overall execution efficiency. Given a set of tasks (with known CPU and memory requirements) and set of compute nodes, we can use the composite model to determine the best node (in terms of thread execution efficiency) and assign the task to the best efficient node to achieve fastest processing.

It is necessary to have accurate models for estimating CPU and memory resources because of the dynamic nature of computer systems and their workload. A number of scenarios with real-world benchmark programs containing dynamic behaviour (regarding CPU and memory requirements) are carried out to measure the accuracy of CPU and memory prediction models. Confidence interval and moving average statistics from measured CPU and memory availability validate the utility of these models. In addition, an extensive empirical study is included to measure the accuracy of the proposed composite prediction and the average CPU prediction models as well.

The rest of this paper is organised in the following manner. Section 2 discusses relevant background related to the CPU availability and memory models, first introduced and motivates the importance of predicting resource availability. Section 3 presents the theoretical derivations of upper- and lower-bounds of the CPU availability model, case study measurements for single- and multi-core CPUs, statistical analysis of the results, and the new average prediction model. The theoretical derivations of upper- and lower-bounds for memory availability are introduced in Section 4. This section also includes the empirical studies

including benchmarking, case study measurements for multi-core CPUs, and statistical analysis of the results. In Section 5, we introduce the composite prediction model and derive the upper- and lower-bounds from the CPU availability and memory model along with empirical studies. Finally, Section 6 contains concluding remarks, application areas of the introduced prediction models, and suggestions for future work.

2 Relevant work

Existing prediction models generally assume CPU resources are equally distributed among all processes in the run queue by following a Round Robin (RR) scheduling technique (Beltrán et al., 2008; Avi et al., 2010). These models use the number of processes in the run queue as the system load index. As a result, the CPU availability prediction for a newly arriving process when there are currently n processes in the run queue is simply $1/(n + 1)$. This predictor is only accurate for CPU-bound processes, which share CPU resources in a balanced manner; consistent with the RR model assumption. But, when the processes also require I/O resources, this approach fails to provide accurate predictions and incurs large prediction errors. Thus, when there are processes in the run queue that require CPU and I/O resources, a more complex model is necessary to describe how the CPU is shared (Beltrán and Guzmán, 2009). The introduced models overcome this limitation and are suitable for both CPU and I/O bound processes.

Fedorova et al. (2007) also worked on operating system scheduling on heterogeneous core systems. They proposed thread-to-core assignment algorithms that optimise performance and demonstrate the need for balanced core assignment. The paper makes the case that thread schedulers for multi-core systems in a heterogeneous environment should target the following objectives: optimal performance, core assignment balance, response time, and fairness. In addition, Dwyer et al. (2012) introduced a practical new method for estimating performance degradation on multi-core processors, and its application to workloads of clusters nodes.

Yeh et al. (2014) have worked on multi-core system and the allied software parallelisation technique trends of system on chip (SoC) design. This paper adopts an electronic system-level (ESL) design methodology for higher system performance and lower energy utilisation for revealing a system performance prediction and analysis method for multi-core systems. Based on the scalable multi-core virtual platform, they have performed one to eight-core system performance trend prediction as well as multi-core system performance analysis. Experiments are conducted for observing the performance improvement of software parallelisation. This paper also discusses regarding hardware-software co-design and the hardware cost reduction.

Zhang et al. (2014) proposed two fault-tolerant scheduling methods on multiprocessor systems through active and passive backup copies. The proposed model uses

integer linear programming and heuristic algorithm for obtaining optimal results (polynomial run-time). The empirical tests conducted in this paper evaluate the proposed methods in terms of scheduling length for a set of DAG benchmarks and shows the effectiveness of proposed methods.

3 CPU availability model

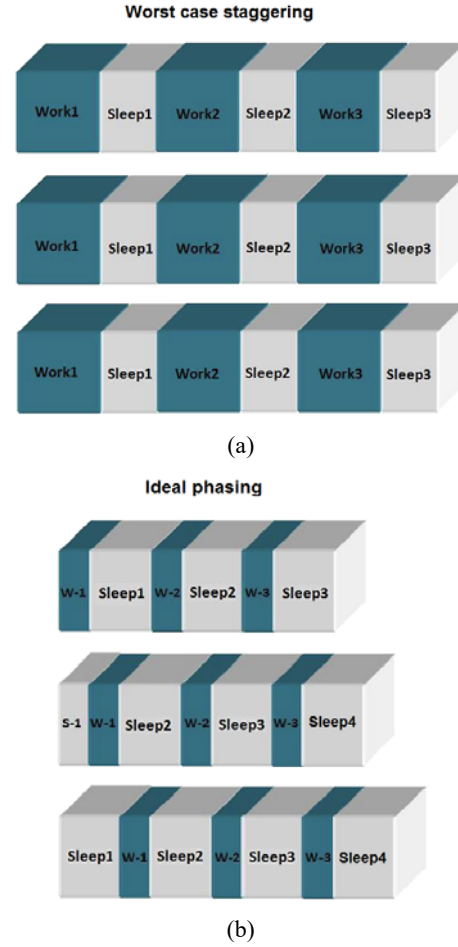
The primary focus of this section is to present a CPU availability prediction model for estimating CPU availability for a set of tasks on multi-core systems. When the number of threads assigned to a multi-core processor is less than or equal to the number of CPU cores associated with the processor, the efficiency of the CPU is often near to ideal. When the number of assigned threads is more than the number of CPU cores, the resulting CPU performance can be more difficult to predict (Hasan et al., 2014). For example, assigning two CPU-bound threads to a single core results in CPU availability of about 50%, meaning that roughly 50% of the CPU resource is available for executing either thread. Alternatively, if two I/O-bound threads are assigned to a single core, it is possible that the resulting CPU availability is nearly 100%, provided that the usage of the CPU resource by each thread is fortuitously interleaved. However, if the points in time where both I/O bound threads do require the CPU resource overlap (i.e., they are not interleaved), then it is possible (although perhaps not likely) that the CPU availability of the two I/O bound threads could be as low as 50%. Therefore, considering the number of processes in the run queue itself as a load index is not sufficient. The aggregate CPU load (sum total) of running threads needs to be considered for deriving an accurate model.

In the framework described here, the execution of a thread is modelled by a series of alternating work and sleep phases. A thread in a work phase will remain there until it has consumed enough CPU cycles to complete the allotted work of that portion of computation. After completing the work phase, the thread then enters in the sleep portion where it stays (does not consume CPU cycles) for a specific amount of time. Each thread is parameterised by a CPU usage factor (Hasan, 2011). The *CPU usage factor* is defined as the time required to complete the work portion of a work-sleep phase on an unloaded CPU, divided by the total time of a work-sleep phase. A thread having zero sleep length has a CPU usage factor of 100%, which is also called a CPU-bound thread. Sleep phase length and the total amount of computational work to accomplish is computed based on the CPU usage factor. Its sleep length relative to that of the work portion based on CPU load factor is used to account for I/O or any other interruptions (Hasan et al., 2014).

The results presented in Hasan et al. (2011) show that n running threads having similar work-sleep phases often result in $1/n$ CPU availability in a single-core machine. In such cases, the work phases get n times longer due to CPU contention among running threads, depicted in Figure 1(a).

Alternatively, if the work portions of threads are staggered (interleaved) to where there is no overlap, by phase shifting, then there is no contention for the CPU resource and the CPU availability is essentially 100%, as shown in Figure 1(b). The ideal phasing illustrated in Figure 1(b) requires that the work of any two sleep threads can be accomplished within the time of one sleep portion of a phase.

Figure 1 Three concurrent threads in a single-core machine with identical work-sleep phases and CPU usage factors, (a) with contending work portions; (b) with interleaved work portions (see online version for colours)



3.1 CPU availability model upper- and lower-bounds

For a single core machine, the following model defines the upper-bound for CPU availability. Here, L is the aggregate loading factor, defined as the sum of CPU usage factors of all threads in a batch.

$$\bar{c} = \frac{1}{\max(1, L)} = \begin{cases} 1, & \text{if } L \leq 1 \\ 1/L, & \text{if } L > 1 \end{cases} \quad (1)$$

The upper-bound model represents the best case CPU availability, which is realised by the example of Figure 1(b) in which none of the threads use the CPU resource concurrently. Provided that the sum of the usage factors of the threads is less than unity, then it is possible that the CPU

availability could be as high as unity (i.e., 100%). When the sum of the CPU usage factors is greater than unity, then the best possible value for CPU availability is $1/L$.

For a single core machine, the following model defines the lower-bound for CPU availability. Here, n is the number of threads assigned to the single core machine.

$$\underline{c} = \frac{1}{1 + \left(\frac{n-1}{n}\right) \times L} \quad (2)$$

The lower-bound model is associated with a situation in which the threads' usage of the CPU resource has maximum overlap, as depicted in Figure 1(a), where all threads of the batch uses the CPU resource concurrently. The execution overhead due to work phase overlap increases when the number of concurrent threads in a batch increases or when the aggregate CPU load among concurrent threads increases. This additional overhead is estimated by the expression $\left[\frac{(n-1)}{n} \times L\right]$ in equation (2) which relies on both the number of concurrent threads and aggregate CPU load of the batch.

For a multi-core machine with r cores, the following models define upper- and lower-bounds for CPU availability. The upper-bound model is:

$$\bar{c} = \frac{1}{\max\left(1, \frac{L}{r}\right)} = \begin{cases} 1, & \text{if } L/r \leq 1 \\ r/L, & \text{if } L/r > 1 \end{cases} \quad (3)$$

and the lower bound is:

$$\underline{c} = \frac{1}{1 + \left(\frac{n-1}{n}\right) \times \left(\frac{L}{r}\right)} \quad (4)$$

Note that equations (3) and (4) are generalisations of equations (1) and (2), i.e., for the case $r = 1$, equations (3) and (4) are identical to equations (1) and (2).

The difference between upper- and lower-bounds can be significant for moderate values of aggregate loading. The difference for a multi-core machine can be as high as 0.446 for eight-threads with aggregate CPU loading of 0.50. To address this challenge, based on the possibility of having a tighter lower-bound, a sharpened CPU availability lower-bound model for multi-core systems is introduced below. The sharpened lower-bound model is derived from the previously introduced lower-bound model in this section.

As described earlier, the lower-bound model always considers the worst case staggering of threads which leads to intense performance degradation due to contention among all running threads. But, it has been observed from the empirical results that when the threads are lightly loaded (i.e., small L), the probability of this worst case staggering is low. That is, three threads in a batch having an aggregate CPU load of 0.3 has a lower probability of worst case staggering compared with three threads with aggregate CPU load of 0.9.

The new lower-bound model consists of two scenarios. In the first scenario, the aggregate CPU load value (L) is less than or equal to the number of CPU cores (r). Threads are lightly loaded in this scenario and the probability of worst case staggering is small. The following estimated value, denoted by μ , is subtracted from the denominator ($L \leq r$) of the equation (4) to provide less penalty. The new parameter to this model is the *core hyper-threading*, defined as ζ (cores can have two or more hardware threads).

$$\mu = \left(\frac{L}{n \times \zeta}\right) \quad \text{if } L \leq r \quad (5)$$

In the second scenario, the value of $L > r$. The CPU load of threads in the batch increases as the value of L increases. As threads' CPU requirement increases, the probability of worst case staggering increases. When the value of L equals to n (i.e., threads become CPU bound), the worst case staggering becomes obvious and the following equation goes to 0. The estimated value, denoted by μ , is subtracted from the denominator ($L > r$) of the equation (4) to provide less penalty when $L < n$.

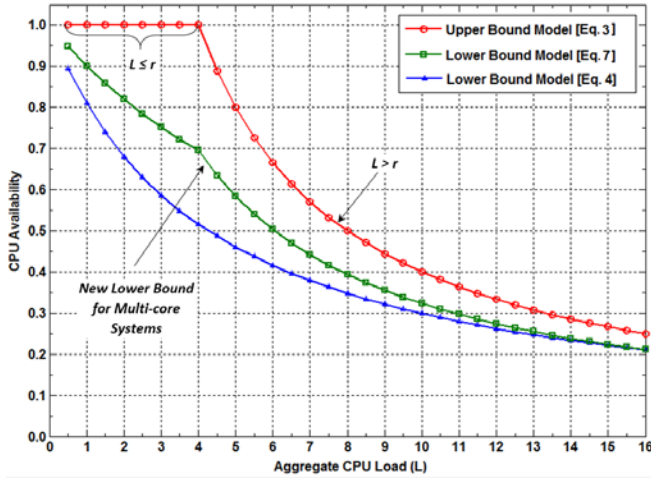
$$\mu = \left(\frac{n-L}{n+r \times \zeta}\right) \quad \text{if } L > r \quad (6)$$

That is, based on equation (6), when the value of $L = n$, the worst case staggering possibility is certain and nothing will be subtracted from the denominator (the sharpened lower-bound model value will be same as the previous lower-bound model). By subtracting the above equations from the introduced lower-bound model, we get the following sharpened lower-bound model.

$$\underline{c} = \begin{cases} \frac{1}{1 + \left(\frac{n-1}{n}\right) \times \left(\frac{L}{r}\right) - \left(\frac{L}{n \times \zeta}\right)} & \text{if } L \leq r \\ \frac{1}{1 + \left(\frac{n-1}{n}\right) \times \left(\frac{L}{r}\right) - \left(\frac{n-L}{n+r \times \zeta}\right)} & \text{if } L > r \end{cases} \quad (7)$$

Figure 2 plots the upper- and both lower-bound models for the case $r = 4$ and $n = 16$. In this figure, the horizontal axis represents aggregate CPU load of the running threads (in a scale of 0 to n) and the vertical axis represents CPU availability (in a scale of 0.0 to 1.0). In an unloaded system, when the aggregate CPU load for running threads are below or equal to the number of CPU cores, the availability is 1.0 (representing 100%) because every thread gets sufficient CPU resources. When the aggregate CPU load reaches above the number of CPU cores, CPU availability to threads decreases resulting in efficiency degradation. Note that the difference in upper- and lower-bounds becomes significantly tight for moderate values of aggregate loading for the new lower-bound model. In the following section, experimental studies are performed to determine actual measured values of CPU availability in relation to these bounds.

Figure 2 Upper- and lower-bounds for CPU availability prediction for $r = 4$ and $n = 16$ (see online version for colours)



3.2 Experimental studies

3.2.1 Overview

The purpose of the experimental studies is to empirically measure the CPU availability as a function of aggregate loading. For the empirical study, real-world benchmark programs presented in Table 1 are developed and utilised. The Monte Carlo π estimation, fast Fourier transformation, and Super prime number generator are compute intensive benchmark programs, thus suitable for validating the CPU availability model. The measured *CPU availability* associated with a collection of threads executing on a processor is defined by the ratio between the ideal time required to execute a benchmark thread (i.e., *piEstimation*, *fft*, or *supPrime*) on an unloaded processor divided by that thread's execution time on a loaded processor. About 2,000 independent test cases are conducted using these compute-intensive benchmark programs to satisfy all possible situations of the CPU model. Uniform sampling of data across the values of possible aggregate CPU loading has been ensured. A test run of each batch of threads provide one measurement (the minimum efficiency is considered for accuracy) of CPU availability. Algorithm 1 presents major parts of the experimental system.

To ensure a uniform sampling of data across the values of possible aggregate loadings, a random value of aggregate loading between $(\epsilon \times n)$ and n is chosen first. The value of $\epsilon = 0.05$, denoting a 0.5% CPU-load, is used to represent the extreme lower CPU load value for a thread. A thread can not have CPU load value of 0.0, else it would never complete its assigned work. The selected aggregate load (*sum-total*) is then distributed among threads using expressions inside the inner for loop. For example, if a thread batch contains eight threads then the min-limit is (0.05×8) 0.4 and the max-limit is 8.0 (i.e., all CPU-bound threads). Then a random CPU load value between 0.4 and 8.0 is chosen and distributed among 8 threads using Algorithm 1.

Algorithm 1 Aggregate load distribution and measurement of execution efficiency

Input: Number of threads (n), and number of runs (Γ)

for count $\leftarrow 1 \dots \Gamma$ **do**

 Select an aggregate CPU load, $L \in [(\epsilon \times n) \dots n]$

for $i \leftarrow 1 \dots (n - 1)$ **do**

$$L_i = \text{Max}\left(L - \sum_{j=1}^{i-1} (T_j - (n - i)), \epsilon\right)$$

$$U_i = \text{Min}\left(L - \sum_{j=1}^{i-1} (T_j - (n - i) \times \epsilon), 1.0\right)$$

 Select T_i randomly so that $T_i \in [L_i, U_i]$

$$T_i \leftarrow (U_i - L_i) \times T_i + L_i$$

end for

$$T_n \leftarrow L - \sum_{i=1}^{n-1} T_i$$

 Compute sleep phase length for each thread [equation (8)]

 Compute total work amount for each thread [equation (9)]

 Assign phase shift values of all threads

 Spawn *benchmark* threads into the system

 Wait for threads to complete assigned work

 Collect and Persist data in respective CSV files

end for

Close all files and connections

Output: Thread execution report

Table 1 Benchmark programs used for validating introduced prediction models

Name	Description	Applied
<i>piEstimation</i>	Uses Monte Carlo method to estimate the value of π	CPU availability model
<i>fft</i>	Fast Fourier transformation	CPU availability model
<i>supPrime</i>	Generates high order prime numbers	CPU availability model
<i>bst</i>	Builds a binary search tree to be sorted	Memory model
<i>denseMatMul</i>	Dense matrix multiplication for scientific computation	Memory model
<i>iRender</i>	Image rendering of large BMP files using image smoothing algorithm	Memory model
<i>bsn</i>	Bitonic sorting network	Composite prediction model
<i>smvm</i>	Sparse Matrix Vector Multiplication	Composite prediction model
<i>tridSolver</i>	Tridiagonal solver	Composite prediction model

The expressions of U_i and L_i is introduced to provide an upper- and lower-limit of available CPU load for the i^{th} thread. A random CPU load value is selected from the range (L_i, U_i) and assigned to the i^{th} thread. The CPU load for the

thread is then scaled and placed into T_i . For example, for a scenario having two threads, a value of aggregate loading is chosen between a small value ($\epsilon \times 2$) and 2.0; denote this value as L . Then a random value is chosen between $\max\{\epsilon, (L - 1.0)\}$ and $\min\{1.0, L\}$, which defines the CPU usage factor of the first thread, say T_1 ; the CPU usage factor of the second thread is then defined as $T_2 = L - T_1$. In general, for n threads, Algorithm 1 is used to randomly assign the CPU usage factors for a given value of aggregate loading L .

Based on CPU load of each thread, total amount of work (upper range) and sleep phase length values are derived. During each work phase, threads accomplish a fixed amount of work. Threads need to run several phases to complete the total amount of work. As described in Algorithm 1, a phase shift measured value is also assigned to each thread ranging from 0 to length of the phase which provides a degree of staggering of phases among the threads.

3.2.2 Experimental environment

The system used for evaluating the single core test cases is equipped with an Intel Xeon E5540 processor, 2.53 GHz clock speed, 1,333 MHz bus speed, and 4 GB of RAM. This machine has Linux kernel version 3.2.0-36. The system used for evaluating the multi-core test cases is equipped with Intel(R) Xeon(R) Quad-core W3520 processor, 2.67 GHz clock speed, 1,333 MHz bus speed, and 6.0 GB of RAM. This machine also has Linux kernel version 3.2.0-36. The average CPU load (represents the average system load over a period of time) was 0.0161302 per core in a scale of 1.0 (in a 15 minute period) before running test cases which indicates that the machines were lightly loaded (essentially unloaded). Owing to the different configurations of the single- and quad-core systems, benchmarking for the single-core and quad-core machines were determined separately. The C programming language in Linux environment is used to implement the analytical model framework and benchmark programs. The gcc compiler version used is 4:6:3. Threads deployed here are independent tasks, meaning there are no interdependencies among threads such as message passing. Threads are spawned concurrently in a single- and multi-core machine with measured work load. When the batch of threads complete assigned work and terminates, an execution report is produced, which contains start time, work time, idle time, end time, number of phases, and others for statistical analysis.

3.2.3 Benchmarking

Benchmarking a thread on an unloaded system enables the calibration of parameters associated with the work and sleep portions of the phases to synthesise a particular CPU usage factor. For setting up the benchmarks in Table 1, each thread is assigned a *total work* and *sleep phase* duration depending on the assigned CPU load value discussed in Algorithm 1. In the work portion of each phase, threads accomplish a fixed amount of CPU intensive work for

generating prime numbers (based on the benchmark program). Depending on the work phase completion time (which is mostly constant for a machine) and CPU load, the sleep phase length of the thread is calculated using the following equation:

$$Sleep = \left(\frac{WorkPhaseTime \times (1.0 - CPUload)}{CPUload} \right) \quad (8)$$

According to equation (8), the sleep time increases as the CPU usage decreases. Thus, a thread with low CPU usage will sleep longer than other threads having higher CPU usage factors. Though the work portion length of threads are same, due to sleep phase length variation due to CPU load, the total phase length (work + sleep) is different for threads. The total amount of work that needs to be accomplished by threads are defined by equation (9). In a time frame, say 10 seconds, the amount of work a thread with higher CPU load can accomplish is more than the amount of work a thread with lower CPU load. Thus, it is needed to calculate the variable amount of work, denoted by ω , required to accomplish by threads to finish assigned work and terminate at the same time.

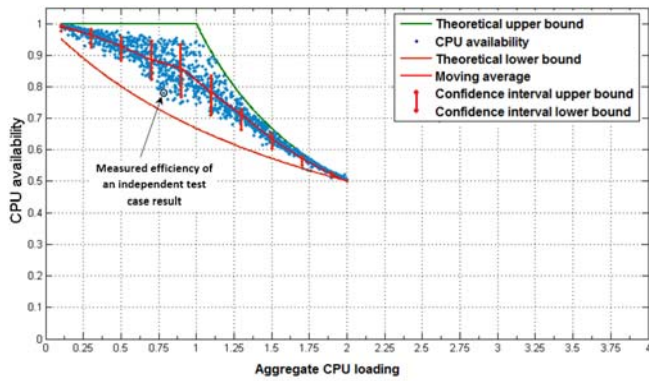
$$\omega = \left(\frac{MaxPhaseTime}{PhaseTime\ of\ T_i} \times (StageCount \times SW) \right) \quad (9)$$

Parameters of equation (9) are the maximum phase time (work + sleep phase time) among all threads, phase time of the thread, number of stages to complete the ideal work, and amount of accomplished work in each phase (SW). Measured total workloads are then assigned to respective benchmark threads so that the batch terminates at the same time for accuracy.

3.2.4 Empirical CPU availability case studies

For measuring the CPU availability of the single-core processor, three case studies were conducted in which multiple (2, 3, and 4) threads were spawned concurrently. An aggregate CPU load L was selected randomly, and distributed among threads as described in Algorithm 1.

Figures 3 and 4 show measured CPU availability scatter graphs for two and four concurrent threads executing on the single-core processor, superimposed with the plots of the upper- and lower-bound models derived in Section 3.1. In these figures, the horizontal axis represents aggregated CPU load and the vertical axis represents CPU availability. Each small dot in these graphs is an independent test case measurement of CPU availability. There are 2,000 dots in each figure representing measured CPU availability value among the concurrent threads. A moving average line is also drawn through the data on the graphs for helping to visualise the average measured performance. A window size of 0.10 aggregate CPU load and incremental value of 0.01 was used to calculate the moving average values. A similar sliding window approach was employed to calculate the 90% confidence interval upper- and lower-limits.

Figure 3 CPU availability of two threads in a single-core machine (see online version for colours)

Note: Results of 2,000 test cases with upper- and lower-bounds and 90% confidence interval bars.

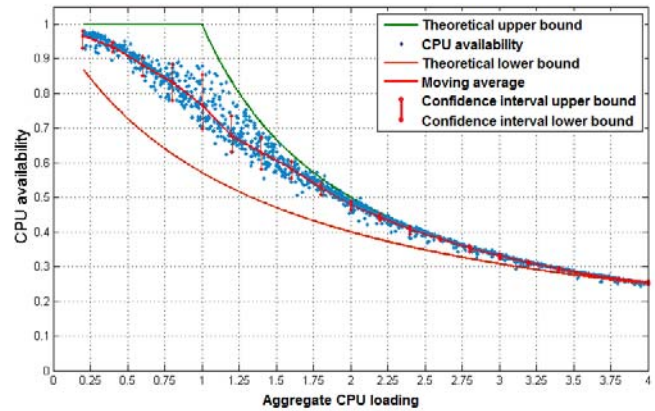
It is apparent from Figures 3 and 4 that the variation in CPU availability is low when the aggregate CPU loading is either relatively low or relatively high. Thus, CPU availability prediction is quite accurate when the CPU is either lightly or heavily loaded. When the total CPU loading is moderate, the measured CPU availability has more variation and thus it is less predictable. One of the intuitive reasons for low variation when threads have small CPU loading factors is their sleep phase lengths are wider, which decreases the probability of work portion overlap among the threads. On the other hand, when the aggregate CPU load is large, threads have smaller sleep phase and longer work phase lengths which almost always forces work phases to overlap and decrease performance.

For measuring the CPU availability of the quad-core processor, three case studies were conducted in which multiple (eight, 12, and 16) threads were spawned concurrently to relatively compare the performance between single- and quad-core processors. A similar approach has been employed to calculate the moving average and 90% confidence interval values. CPU availability graphs for eight and 16 threads are shown in Figure 5 and Figure 6 respectively. Figure 5 shows a lower variation of CPU availability compared with Figure 3, which shows CPU availability of two threads in a single-core machine. This decrease in variation may be explained as follows: During the thread execution life cycle, depending on CPU availability, threads might be allocated in different cores for load balancing which could decrease the probability of work phase overlap.

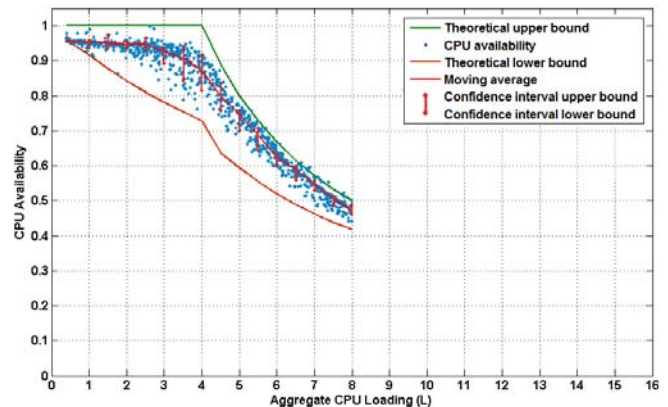
The empirical results for the quad-core processor also show that the CPU availability prediction is quite accurate when the CPU is either lightly or heavily loaded. When the total CPU loading is moderate, the measured CPU availability has more variation and thus it is less predictable.

In further reporting the results of the studies, it is convenient to define the normalised aggregate load, L/n , which is the aggregate load L normalised by the number of threads n . For sample values of normalised aggregate load, Table 2 shows the average measured CPU availability (Avg.), the difference in the upper- and lower-bound models

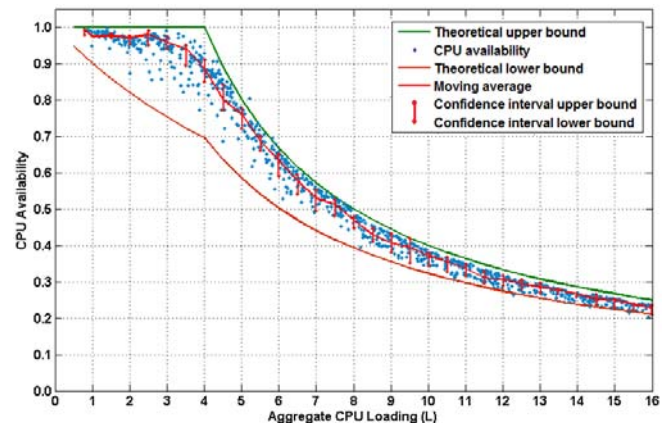
(BD) and the difference in the 90% confidence interval limits (CI Diff) for two, three, and four concurrent threads on the single-core processor.

Figure 4 CPU availability of four threads in a single-core machine (see online version for colours)

Note: Results of 2,000 independent test cases, bounds, and 90% confidence interval bars.

Figure 5 CPU availability of eight threads in a quad-core machine (see online version for colours)

Note: Results for 2,000 independent test cases and 90% confidence intervals.

Figure 6 CPU availability of 16 threads in a quad-core machine (see online version for colours)

Note: Results of 2,000 independent test cases and 90% confidence intervals.

Table 2 CPU availability data for two, three, and four threads in a *single-core machine* consists of average measured CPU availability (Avg), differences of bound models (BD), and 90% confidence interval limits (CI Diff)

L/n	2 thread CPU availability			3 thread CPU availability			4 thread CPU availability		
	Avg	BD	CI Diff	Avg	BD	CI Diff	Avg	BD	CI Diff
0.05	0.991	0.048	0.019	0.978	0.094	0.026	0.966	0.133	0.048
0.10	0.978	0.091	0.046	0.955	0.167	0.047	0.933	0.232	0.028
0.20	0.946	0.167	0.071	0.889	0.287	0.093	0.831	0.375	0.105
0.30	0.907	0.231	0.118	0.794	0.376	0.144	0.677	0.306	0.106
0.40	0.871	0.286	0.167	0.726	0.277	0.131	0.582	0.170	0.051
0.50	0.819	0.333	0.175	0.648	0.167	0.057	0.478	0.099	0.025
0.60	0.746	0.208	0.122	0.547	0.101	0.041	0.407	0.059	0.019
0.70	0.672	0.126	0.058	0.465	0.059	0.026	0.353	0.035	0.018
0.80	0.602	0.069	0.038	0.411	0.032	0.019	0.313	0.018	0.008
0.90	0.546	0.029	0.013	0.361	0.013	0.011	0.273	0.008	0.006
1.00	0.500	0.000	0.003	0.331	0.000	0.005	0.249	0.000	0.007

Table 3 CPU availability data for eight, 12, and 16 threads in a *quad-core machine* consists of average measured CPU availability (Avg), differences of bound models (BD), and 90% confidence interval limits (CI Diff)

L/n	8 thread CPU availability			12 thread CPU availability			16 thread CPU availability		
	Avg	BD	CI Diff	Avg	BD	CI Diff	Avg	BD	CI Diff
0.05	0.977	0.036	0.023	0.973	0.059	0.019	0.971	0.080	0.014
0.10	0.962	0.070	0.021	0.959	0.111	0.022	0.963	0.149	0.021
0.20	0.951	0.130	0.046	0.946	0.200	0.054	0.943	0.259	0.046
0.30	0.946	0.184	0.034	0.887	0.273	0.062	0.781	0.230	0.054
0.40	0.917	0.231	0.102	0.762	0.259	0.079	0.587	0.149	0.052
0.50	0.879	0.273	0.113	0.621	0.185	0.096	0.469	0.107	0.047
0.60	0.754	0.224	0.079	0.493	0.141	0.085	0.395	0.081	0.066
0.70	0.681	0.168	0.063	0.433	0.112	0.073	0.326	0.065	0.057
0.80	0.587	0.130	0.056	0.361	0.092	0.054	0.289	0.054	0.026
0.90	0.526	0.103	0.037	0.329	0.078	0.039	0.254	0.046	0.028
1.00	0.471	0.083	0.028	0.301	0.067	0.027	0.228	0.039	0.024

Table 2 shows that the difference between the upper- and lower-bounds can reach as high as 0.375, for four threads and a normalised aggregate loading of 0.20. However, the measured 90% confidence interval difference for the same case is much smaller, around 0.105. The difference of the formula-based bound is more precise when the CPU is lightly or heavily loaded.

Similarly, for a quad core processor, Table 3 shows that the difference between the upper- and lower-bounds can reach as high as 0.273, for eight threads and a normalised aggregate loading of 0.50. However, the measured 90% confidence interval difference for the case is much smaller, around 0.113. The difference of the formula-based bound is more precise when the CPU is lightly or heavily loaded for quad-core processor as well. Additionally, the empirically-based values for CI Diff can be used as a basis for creating sharper estimates of CPU availability.

3.3 Average CPU availability prediction

This section introduces a model for predicting the expected (on average) availability of CPU (instead of calculating the upper- and lower-bounds). As illustrated in previous

subsection, exact values of CPU availability are difficult to predict because of dependencies on many factors, including context switching overhead, memory speed, CPU usage requirements of the threads, core hyper-threading, the degree of interleaving of the timing of the CPU requirements of the threads, and the characteristics of the thread scheduler of the underlying operating system. Due to the complex nature of the execution environment, an empirical approach is employed here to estimate expected CPU availability.

3.4 The model

Based on the observed shape and values of measured average CPU availability in Figures 5 and 6, and Table 3 of Subsection 3.2.4, the mathematical model for average CPU availability is derived. It can be observed from Figures 5 and 6 that the shape of measured average CPU availability has similarities with the upper bound model [derived in equation (3)].

Because of the similarities with the upper-bound model, the average CPU availability model is also derived based on two scenarios. In the first scenario, when the value of

aggregate CPU load (L) is less than or equal to the number of CPU cores (r), that is, running threads has low CPU utilisation, the likelihood of work phase contention is minimal. This results in less contending work phases among running threads but still some context switching overhead. The CPU availability upper-bound model is based on the situation in which all work portions of threads are phased out (interleaved) but context switching overhead is ignored. The following model represents the context switching overhead when $L \leq r$. This estimated context switching overhead is subtracted from the first scenario of the upper-bound model to reflect the observed behaviour.

$$\left(\frac{n-1}{n+r}\right) \times \left(\frac{L}{(n+r) \times \xi}\right) \quad \text{if } L \leq r \quad (10)$$

In the second scenario, when the value of $L > r$, the best available CPU for running threads is considered as r/L by the upper-bound model. This scenario is based on interleaved work phase of running threads. Though this model provides a good estimation but when the value of L increases, threads become more CPU hungry and the contention is more likely due to relatively wider work portions than idle portions. That is, more CPU load increases more CPU contention and more context switching overhead. The following estimated context switching overhead is subtracted from the second scenario of the upper-bound model to reflect the observed behaviour.

$$\left(\frac{1}{1 + \left(\frac{n-1}{n}\right) \times \left(\frac{L}{r}\right) \times (r \times \xi + \xi)}\right) \quad \text{if } L > r \quad (11)$$

Equation (12), provides a model/explanation of what has been observed for thread assignment in processor on the average in Subsection 3.2.4 (and in Hasan et al., 2014). The model of equation (12) depends on the aggregate CPU load (sum total) of the set of tasks, number of threads, number of processor cores, and number of hyper-threading in cores, denoted by ξ . For a multi-core machine, the following

prediction model estimates the average CPU availability for a set of tasks:

$$c_{avg} = \begin{cases} 1 - \left(\frac{n-1}{n+r}\right) \times \left(\frac{L}{(n+r) \times \xi}\right), & \text{if } L \leq r \\ \frac{r}{L} - \left(\frac{1}{1 + \left(\frac{n-1}{n}\right) \times \left(\frac{L}{r}\right) \times (r \times \xi + \xi)}\right), & \text{if } L > r \end{cases} \quad (12)$$

The model of equation (12) is derived from equations (10) and (11) and depends on whether the aggregate CPU load is less than the number of cores or more than the number of cores. In the first situation, CPU resources are lightly loaded resulting in less context switching overhead and better efficiency. In the second situation, threads are moderate to highly loaded (i.e., aggregate CPU load is more than the number of cores), resulting in more context switching overhead and reduced efficiency. The usage of CPU resource for threads has maximum overlap as depicted in Figure 1(a). In general, a thread with more CPU load incurs more contention for resources and context switching overhead. Therefore, an estimated context switching overhead is subtracted from the efficiency value in both cases (i.e., when for $L \leq r$ or $L > r$) to best-fit the average efficiency plot.

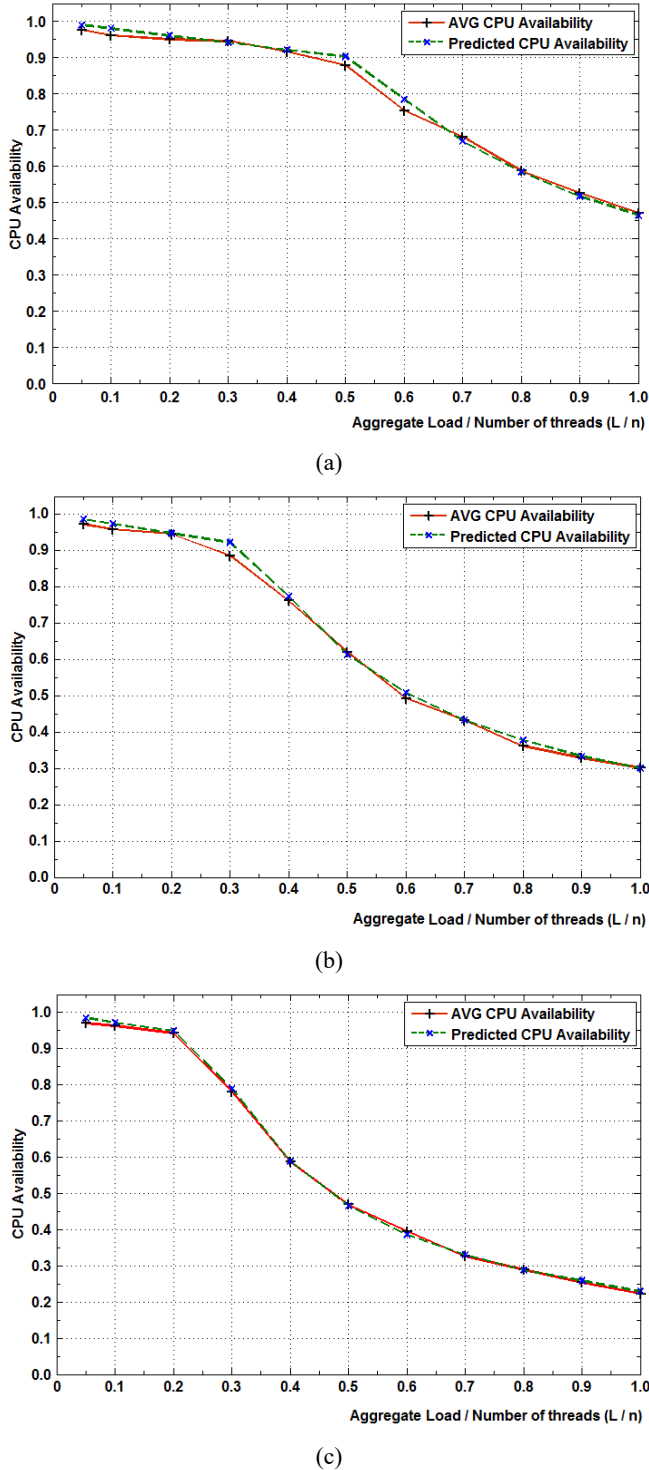
3.5 Average CPU availability model verification

As the average CPU availability model is derived from the Table 3 in Section 3.2.4, which presents the multi-core CPU availability empirical results, *a new set empirical tests* are conducted to validate the accuracy of the model. Figures 7(a), 7(b) and 7(c) correspond to three new independent sets of experiments for 8, 12, and 16 concurrent thread batches in a quad-core machine. In this figure, the horizontal axis represents the normalised aggregate load, L/n , and the vertical axis represents the CPU availability for running threads. Each empirical study consists of 2,000 independent test runs like shown in Figures 5 and 6.

Table 4 Measured prediction error by the CPU availability average model for eight, 12 and 16 threads in a quad-core machine

L/n	8 thread CPU availability			12 thread CPU availability			16 thread CPU availability		
	Avg	Model (cavg)	Error (%)	Avg	Model (cavg)	Error (%)	Avg	Model (cavg)	Error (%)
0.05	0.979	0.990	1.08	0.9714	0.987	1.56	0.9716	0.985	1.34
0.10	0.965	0.981	1.64	0.9653	0.974	0.87	0.9607	0.970	0.93
0.20	0.943	0.961	1.82	0.9408	0.948	0.72	0.9364	0.940	0.36
0.30	0.933	0.942	0.89	0.8942	0.923	2.88	0.7741	0.786	1.19
0.40	0.898	0.922	2.36	0.7377	0.774	3.63	0.5939	0.585	0.89
0.50	0.871	0.903	3.24	0.6288	0.614	1.48	0.4458	0.465	1.92
0.60	0.760	0.785	2.53	0.5054	0.508	0.26	0.3753	0.386	1.07
0.70	0.679	0.669	0.96	0.4137	0.433	1.93	0.3358	0.330	0.58
0.80	0.571	0.583	1.19	0.3857	0.378	0.77	0.2767	0.288	1.13
0.90	0.510	0.517	0.72	0.3235	0.334	1.05	0.2627	0.255	0.77
1.00	0.473	0.464	0.85	0.3038	0.300	0.38	0.2242	0.229	0.48

Figure 7 (a) Validation of the average CPU availability prediction model associated with equation (12) for (a) multi-core systems with $r = 4$ and $n = 8$ (b) multi-core systems with $r = 4$ and $n = 12$ (c) multi-core systems with $r = 4$ and $n = 16$ (see online version for colours)



A similar approach explained in Section 3.2.4 is also applied here to measure the running average value from the dataset. A window size of 0.10 aggregate CPU load and incremental value of 0.01 was used to calculate the moving average values to plot the line. It can be observed that the CPU availability model lines are smooth and follow similar

pattern compared with the average lines obtained empirically. As the number of threads in a batch increases, the shape and pattern of the line of prediction model become almost identical to average measured line.

Table 4 depicts normalised aggregate load, L/n , average measured CPU availability (Avg.), prediction model value (Model), and prediction error in percentage (Error). Table 4 contains the new empirical data to validate the model. It can be observed from the table that for a set of eight threads, the maximum prediction error is 3.24% when normalised aggregate loading is 0.5. For 12 threads, the maximum prediction error is 3.63% when normalised aggregate loading is 0.4. Finally, for 16 threads, the maximum error is 1.92% when normalised aggregate loading is 0.5. This analysis shows that the introduced average CPU availability model is consistent with the new set of data as well. Though the model is created from one set of data, it can accurately predict the average CPU availability for a new and independent set of data as well. The maximum predicted error is only 3.63%. Thus, the model predicts availability accurately and reliability and can be deployed in real-world applications for scheduling.

3.6 Outcome

Section 3 has introduced two prediction models:

- given a set of tasks, aggregate CPU load of the set of tasks, and number of CPU cores of a machine, we can predict the *upper- and lower-bounds* of CPU availability for the set of tasks
- given a set of tasks, aggregate CPU load, number of CPU cores, and hyper-threading of a machine, we can predict the *average CPU availability* for the set of tasks.

An extensive set of empirical studies have been conducted for validating the prediction models in single- and multi-core architectures. Thread availability scatter plots provide clear visualisation of measured performance based on the density of the dots in the plots. These empirically measured availability values are showed to generally fall within theoretically derived upper- and lower-bound models. A 90% confidence interval for measured availability is shown to provide tighter upper- and lower-limits than the theoretically derived models for upper- and lower-bounds. The bounds are necessary since the actual CPU availability changes based on the order of execution of tasks in the batch. Additionally, the prediction model for upper- and lower-bound is consistent with the dynamic nature of system and its workload, which can vary drastically in a short amount of time.

The empirical studies performed for validating the average CPU availability model shows that the model values follow the same shape and pattern of the experimentally measured average CPU availability lines. This model is suitable for applications that require a single prediction value instead of upper- and lower-bounds for dispatching tasks. This reliable and accurate predication

model could be used as part of a scheduling module to determine the order in which tasks should be assigned to the system so that the completion time of all the tasks is minimised.

4 Memory availability model

In addition to CPU availability, the success of approaches for assigning threads (or processes) to multi-core systems relies on the existence of a reasonably accurate memory model for estimating the execution efficiency of tasks due to memory availability. This is because there is a strong relationship between a thread's total execution time and the availability of memory resources used for its execution. "Predicting the execution efficiency of tasks when the available memory is less than required memory is critical in making task assignment and scheduling decisions" (Beltrán et al., 2008; Tanenbaum, 2008).

It is customary to keep several processes running in time-shared systems. For a process to execute, it is necessary for the system to swap in the required portion of code and data of the process into the primary memory. Additional pages are loaded only when they are demanded during program execution. As the degree of parallelism increases, over-allocating memory may result in severe performance degradation. Demand paging saves the I/O necessary to load pages that are never used but increases the probability of memory overrun. If one access out of 1,000 causes a page fault, the system can be slowed down by a factor as high as 40 because of the page fault service time. The effective access time is directly proportional to the page-fault rate (Avi et al., 2010; Tanenbaum, 2008).

Table 5 Terms and definitions of the memory model parameters

Terms	Definition
$R_i > 0$	R_i is the required number of memory frames by the process i where $i = (1, 2, \dots, m)$.
$R > 0$	R is the total required number of memory frames by all processes ($R = R_1 + R_2 + \dots + R_n$).
$M_a \geq 0$	M_a is the total number of free memory frames available in the system.
A_c	A_c is the access time of translation look-aside buffer in the cache memory.
A_p	A_p is the access time of primary memory.
A_v	A_v is the access time of virtual memory (backing store).
$\rho > 0$	ρ is the data process time for the thread.
B_l	B_l is the backing store latency.
B_s	B_s is the backing store seek time.
κ	Command queue delay.

4.1 Bounds for memory availability

An analytical framework is developed here based on the availability of primary memory for predicting the efficiency associated with concurrent thread execution in multi-core

machines. The primary contribution of this section is the derivation of upper- and lower-bound formulas for the memory model. Table 5 summarises the notation and definitions of required parameters of memory model bounds. The memory upper-bound model consists of two scenarios. When M_a (free memory frames available) is greater than or equal to R (required memory frames) and the cache memory hit ratio is 100% (all page entries are in the translation look-aside buffer), there will be a single look-up in the page table and no additional virtual memory access overhead due to page fault service time. The second scenario is where M_a is less than R ; owing to shortage of required number of memory frames, pages will be swapped out to virtual memory (Grounds et al., 2009; Shrestha et al., 2009). For deriving the upper-bound efficiency model, the ideal thread execution time, denoted by τ [derived in equation (14)], is divided by an expression that represents the execution time, which incorporates the virtual memory access time and backing store overhead for the swapped pages.

The upper-bound model, denoted by $\bar{m}$, for memory efficiency can be represented as:

$$\bar{m} = \begin{cases} 1 & \text{if } M_a \geq R, \\ \frac{\tau}{\tau + (A_c + A_v) \times \left(\left(\sum_{i=1}^n R_i \right) - M_a \right) + (B_l + B_s)} & \text{else.} \end{cases} \quad (13)$$

The ideal thread execution time (τ) represents the best possible execution environment in which threads receive the required amount of primary memory in all situations and the cache memory hit ratio is 100%. That is, for each page, there is only one look-up in page table and a single access in primary memory and data are available in the primary memory. Thus, the ideal memory access time and the data process time, ρ , represents the ideal thread execution time.

$$\tau = \left\{ (A_p + A_c) \times \sum_{i=1}^n R_i \right\} + \rho \quad \text{if } M_a \geq R \quad (14)$$

Figure 8 Upper-bound efficiency curves associated with equation (14) for various ideal thread execution time τ in sec (see online version for colours)

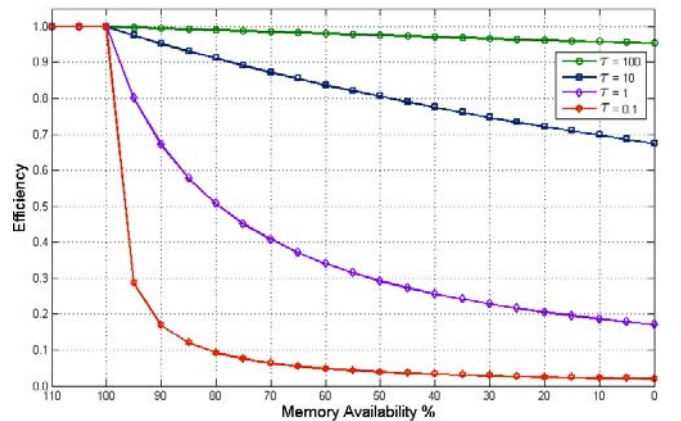


Figure 8 illustrates a collection of plots for the theoretically derived upper-bound model (m) for different ideal thread execution times (τ). In this figure, the horizontal axis represents the *memory availability percentage* (the percentage of available primary memory with respect to the required amount of memory by threads) and the vertical axis represents the efficiency of thread execution associated with equation (13). It can be observed from Figure 8 that when the memory availability of the machine decreases, efficiency of the machine for process execution decreases due to increased activity by the pager (handling page faults). It can also be observed that when the value of τ is small (i.e., process runs for a short period of time), the efficiency decrease is more dramatic compared with large values of τ . For example, if two processes require same amount of memory but process one (P_1) runs for a shorter period of time compared to process two (P_2), due to same page fault service time overhead, P_1 has relatively more efficiency degradation compared with P_2 , which runs for a longer period of time.

Algorithm 2 Setting up the memory framework for the benchmark programs by allocating and holding memory

Input: Number of threads (n), Number of test runs (tr), Memory requirements of each thread (ts) in KB, and memory availability percentage (MAP).

for count $\leftarrow 1 \dots tr$ **do**

 Select a random MAP between 0 ... 120:

$R \leftarrow \frac{\sum_{i=1}^n ts_i}{\text{page size}}$, sizes are in KB.

$M_a \leftarrow \frac{\text{total memory} - \text{used memory}}{\text{page size}}$ of the system.

$cm \leftarrow M_a - (R \times MAP) \times 0.01$

 Create child threads to carryout following tasks:

for $k \leftarrow 1 \dots cm$ **do**

 Allocate memory, $m = (\text{void} *) \text{malloc}(\text{page size})$

 Initialise pages, $\text{memset}(m, 0, \text{page size})$

end for

 Hold the memory until all threads terminate

 Generate and spawn threads concurrently for measuring execution efficiency (Algorithm 3).

end for

Output: R , M_a , and allocated memory.

The lower-bound model represents the worst-case memory access time efficiency that additionally includes a primary memory access time because of cache miss hit, the backing store seek time for individual pages (virtual memory access by multiple concurrent threads will result in storage of pages in non-consecutive order), and the I/O queuing delay. The lower bound model, denoted by $\underline{m}$, additionally includes the backing store overhead expressed in equation (16) and cache ‘miss hit’ overhead, can be represented as:

$$\underline{m} = \begin{cases} \frac{\tau}{\tau + A_p \times \sum_{i=1}^n R_i}, & \text{if } M_a \geq R, \\ \frac{\tau}{\tau + (A_p \times M_a) + (A_v + A_p + A_c)} \times \left(\left(\sum_{i=1}^n R_i \right) - M_a \right) + \eta, & \text{else.} \end{cases} \quad (15)$$

The backing store overhead, denoted by η , can be expressed as:

$$\eta = \left(B_l + \sum_{M_a+1}^R (B_s) + \kappa \right). \quad (16)$$

The backing store overhead (η) increases based on the number of page swap (in/out) and page seek time. The difference in upper- and lower-bound models might be significant, when the initial page fault starts, owing to the uncertainty of the backing store seek time, latency, and command queue delay, and others. The number of disk commands waiting in the queue is normally the factor that slows down the disk performance by increasing the average disk queue time (Hasan et al., 2011; Tanaka, 2005).

4.2 Experimental studies

4.2.1 Overview

The purpose of these experimental studies is to measure the efficiency of thread execution as a function of memory availability for collections of threads in an actual dynamic environment. For the study, two sets of programs have been developed. Algorithm 2 presents the high-level pseudo-code for the first set of programs. The primary role of the first set of programs is to generate threads to allocate and initialise primary memory depending on the memory available percentage and hold the memory until all spawned benchmark threads terminates.

The second set of programs are the memory benchmark programs presented in Table 1. The *binary search tree*, *dense matrix multiplication*, and *image rendering* programs are selected as benchmark programs because these programs include memory intensive expressions and require reasonable amount of free memory for processing data. Required data are generated randomly and assigned to one or two dimensional array depending on the benchmark program. Algorithm 3 presents a high-level pseudo-code for creating and spawning the memory benchmark programs.

The Linux kernel creates a new virtual address space for each child thread because threads are created by using the `fork()` system call. The kernel creates a complete copy of the existing process’s virtual address space; then it copies the parent process’s `vm_area_struct` descriptors and will create a new set of page tables for the child (Avi et al., 2010; Tanenbaum, 2008). The parent’s page table and references will be directly copied; thus the parent and child shares the same physical pages in their address space for all test cases.

4.2.2 Experimental environment

The same experimental environment described in Section 3.2.2 is used for the memory studies. As the purpose of these tests is to precisely measure the effect of memory availability on thread execution, the number of concurrent threads generated using benchmark programs spawned were always less than or equal to the number of CPU cores to prevent any possible CPU related overhead due to context switching or other CPU-related inefficiencies.

Algorithm 3 Creating and spawning child threads for measuring thread execution efficiency of memory benchmark programs

Input: Number of threads (n), 2D array size (s), pipe descriptor (fd), and linear search key .

Wait for the response of consume memory operation

Generate a child process to govern next set of child threads via pipe communication

for $i \leftarrow 1 \dots n$ **do**

 Create the child thread T_i

 Create communication link, pipe, between threads

 Take the *start time* snapshot for the thread T_i

 Compute s_i with respect to memory req. of the i^{th} thread

 Allocate memory for a large 2D array size of s_i

 Initialise array with random numbers $\in [1..10k]$

 Perform work based on benchmark program

 Release the memory occupied by the array

 Take the *end time* snapshot for the thread T_i

$Execution\ time \leftarrow (start\ time - end\ time)$

$Efficiency \leftarrow \frac{benchmark\ time}{execution\ time}$

 Transfer *Efficiency*, *Execution time*, *utilised mem size*, and *thread number* to its parent via pipe

end for

Wait for all child threads to terminate

Store execution data into respective CSV files

Output: Thread execution efficiency data.

4.2.3 Benchmarking

Each *binSearchTree*, *denseMatMul*, and *iRender* benchmark thread consists of a set of operations that are a mix of CPU and memory related expressions that are ideal for the empirical environment of the memory model. For setting up the benchmark, threads are assigned an independent two dimensional array size of 250 MB. For an array, memory allocation of rows have been completed first followed by the memory allocation of columns of each row. After memory allocation, each memory frame was initialised with a random value ranging from 1 to 10,000 to be searched. This initialisation also helps to avoid fake memory allocation (due to demand paging). The system function, *meminfo*, is invoked to validate the accuracy of memory allocation for respective threads. For the *iRender*

benchmark program, several large bitmap files are rendered using the image smoothing algorithm and the output file is stored in secondary disk. For benchmark runs, available primary memory of the system was above the required memory of threads to ensure 100% availability of primary memory to complete assigned work. This approach ensures no additional overhead due to page fault service. Constants and literals in expressions (for generating work load) are avoided to eliminate caching effects. Machine's efficiency data is stored in multiple CSV files for benchmarking and statistical analysis.

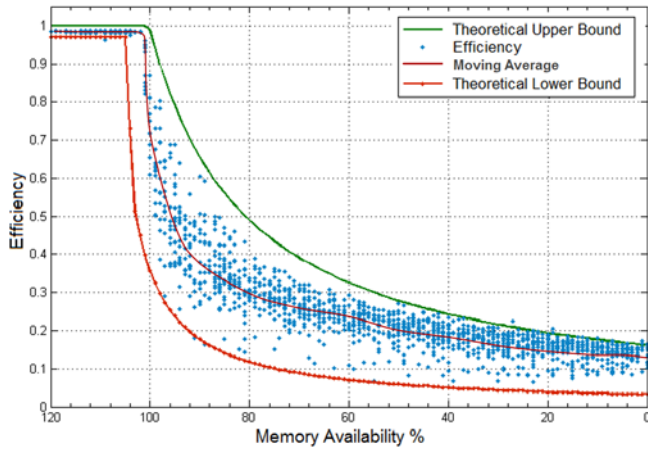
4.2.4 Empirical memory availability case studies

A wide range of empirical studies were conducted to validate the proposed memory availability model of equations (13) and (15). For estimating the effect of memory availability in concurrent thread execution, four major independent case studies were conducted in which one, two, three, and four threads were spawned concurrently in a quad-core machine. Each case study is incorporated with 2,000 independent test runs for measuring the thread execution time where memory availability was varied from 120% down to 0% with respect to the memory requirement of threads.

Figures 9, 10, and 11 show measured thread execution efficiency scatter graphs for one, two, and four concurrent threads, superimposed with plots of the theoretically derived upper- and lower-bound models associated with equations (13) and (15). In these figures, the horizontal axis represents the memory availability percentage (system's memory availability with respect to running threads) and the vertical axis represents efficiency of thread execution (the ideal thread execution time divided by the test run execution time) on a scale of 0.0 to 1.0. Each small dot in these graphs are an independent test case measurement of thread execution efficiency. This empirical study on a quad-core machine shows that as page swap out starts due to lack of primary memory availability, the thread execution efficiency falls sharply because of the page fault service time. The variation in thread execution efficiency is the most, thus less predictable, when the availability is in the range of 100% to 80%. Reasons for wide variation are the initial backing store latency, command queue delay, and serial I/O processing. From the figures, it is apparent that the theoretical upper- and lower-bounds introduced in this paper accurately bind the actual measured values of thread execution efficiency.

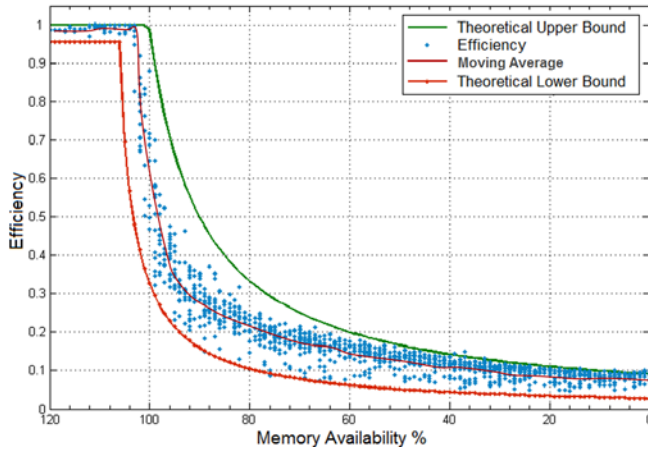
Figure 11 shows a smaller bound difference compared with one and two thread cases. The variation in execution efficiency reduces when the number of threads in a batch increases. It can also be observed that when the memory availability is above 100%, the bound difference increases as the number of threads increases. As the number of threads increases, the probability of page swap out and queuing delay increases. The difference of the model-based bound is more precise when the memory availability for threads is either high or low.

Figure 9 Efficiency of thread execution for one thread in a quad-core machine (see online version for colours)



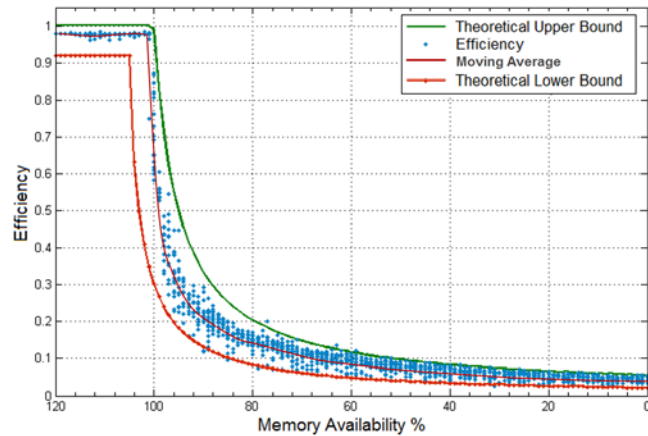
Note: Results of 2,000 independent test cases along with derived upper- and lower-bounds.

Figure 10 Efficiency of thread execution for a set of two threads in a quad-core machine (see online version for colours)



Note: Results of 2,000 independent test cases along with derived upper- and lower-bounds.

Figure 11 Efficiency of thread execution for a set of four threads in a quad-core machine (see online version for colours)



Note: Results of 2,000 independent test cases along with derived upper- and lower-bounds.

In further reporting the results of the studies of one, two, three, and four concurrent threads in a quad-core machine, Table 6 presents the value of theoretically derived upper-bound model (UB), lower-bound model (LB), the difference between upper- and lower-bounds (BD), and the average measures thread execution efficiency (AVG) while the memory availability was varied from 0% to 120%. From the table, it can be observed that the difference between upper- and lower-bounds can reach as high as 0.688 when $n = 4$ and $MAP = 98$ (i.e., when the page fault just starts). However, the measured average thread execution efficiency is for this same case is much smaller, around 0.379. Bound differences are low for the four thread case when the memory availability is relatively high or low.

4.3 Outcome

Analytical models (and empirical studies) were developed for predicting (and measuring) execution efficiency of concurrent threads in Linux environment for multi-core architectures as a function of memory availability. As would be expected, degradation in thread execution efficiency occurs when primary memory availability is less than the total required memory by threads. In addition to memory availability, the total amount of memory required by concurrent threads is a factor in predicting the thread execution efficiency. When the total memory requirement is higher than the total available memory, the relative performance can have a significant impact on execution time. Specifically, increased page swap out by concurrent threads and long queue delay due to serial I/O degrades thread execution efficiency. A memory efficiency scatter plot provides a clear visualisation of measured efficiency based on the density of the dots in the plot. These empirically measured values are showed to generally fall within theoretically derived upper- and lower-bound models.

5 Composite CPU and memory model

The CPU availability and memory models from Sections 3 and 4, respectively, are used in this section to derive a composite CPU/Memory prediction model for estimating the efficiency of thread execution in multi-core systems. As new processes are assigned or existing processes complete execution, the CPU and memory availability of a given machine can change significantly in a short span of time. Existence of the composite prediction model is important because there exists a wide range of applications and scientific models (e.g., geological, meteorological, economical and others) that requires extensive use of both CPU and memory resources, repeatedly. Assigning a set of batch tasks in a distributed environment (distributed schedulers) can utilise the composite model to determine the order (or find subsets) in which the tasks should be assigned to compute nodes so that the completion time for all tasks is minimised prior to its placement in the run queue (Hasan et al., 2013; Garay et al., 2013).

Table 6 Average measured thread execution efficiency (AVG), upper-bound (UB), lower-bound (LB), and differences between upper- and lower-bounds (BD)

MAP (%)	1 thread memory availability				2 thread memory availability				3 thread memory availability				4 thread memory availability			
	AVG	UB	LB	BD	AVG	UB	LB	BD	AVG	UB	LB	BD	AVG	UB	LB	BD
120%	1.000	1.000	0.978	0.022	1.000	1.000	0.957	0.042	1.000	1.000	0.933	0.067	1.000	1.000	0.920	0.081
110%	1.000	1.000	0.978	0.022	1.000	1.000	0.957	0.042	1.000	1.000	0.933	0.067	1.000	1.000	0.920	0.081
100%	0.439	0.991	0.358	0.633	0.401	0.991	0.328	0.663	0.381	0.988	0.327	0.662	0.379	0.992	0.303	0.688
90%	0.382	0.657	0.178	0.479	0.276	0.500	0.160	0.341	0.245	0.391	0.145	0.247	0.214	0.326	0.132	0.194
80%	0.296	0.491	0.118	0.373	0.218	0.335	0.105	0.229	0.182	0.244	0.093	0.151	0.146	0.195	0.084	0.111
70%	0.261	0.392	0.089	0.303	0.167	0.251	0.079	0.173	0.139	0.177	0.068	0.109	0.102	0.139	0.062	0.077
60%	0.247	0.326	0.071	0.255	0.142	0.201	0.063	0.138	0.103	0.139	0.054	0.085	0.083	0.108	0.049	0.059
50%	0.207	0.279	0.059	0.220	0.131	0.168	0.052	0.116	0.084	0.115	0.045	0.070	0.061	0.089	0.040	0.048
40%	0.179	0.244	0.051	0.194	0.115	0.144	0.045	0.099	0.061	0.097	0.038	0.059	0.050	0.075	0.034	0.040
30%	0.161	0.217	0.044	0.173	0.093	0.126	0.039	0.087	0.051	0.085	0.033	0.051	0.044	0.065	0.030	0.035
20%	0.147	0.177	0.039	0.156	0.080	0.112	0.035	0.077	0.047	0.075	0.029	0.045	0.041	0.057	0.027	0.031
10%	0.139	0.177	0.035	0.142	0.071	0.101	0.031	0.070	0.044	0.067	0.026	0.041	0.038	0.051	0.024	0.027
0%	0.134	0.163	0.030	0.130	0.067	0.092	0.028	0.063	0.041	0.061	0.024	0.037	0.031	0.046	0.022	0.025

5.1 Bounds of composite CPU and memory model

A composite analytical framework consisting of upper- and lower-bounds are derived for estimating the overall efficiency for a batch of tasks in multi-core systems. Input parameters for the composite upper- and lower-bound models include the CPU availability upper- and lower-bound models derived in equations (3) and (7), and memory efficiency upper- and lower-bound models derived in equations (13) and (15), respectively. The composite CPU and memory upper-bound model, denoted by $\overline{cm}$, represents the best case efficiency value of a machine for concurrent thread execution can be represented by the product of two models because CPU and memory are the two primary factors used to characterise machines.

$$\overline{cm} = \begin{cases} \frac{1}{\max\left(1, \frac{L}{r}\right)} & \text{if } M_a \geq R, \\ \frac{1}{\max\left(1, \frac{L}{r}\right)} \times \frac{\tau}{\tau + (A_c + A_v)} \times \left(\left(\sum_{i=1}^n R_i \right) - M_a \right) & \text{else.} \end{cases} \quad (17)$$

The values of $\overline{c}$ and $\overline{m}$ represents the relative impact of a machine's overall efficiency due to loading of machine's CPU and memory resources, respectively. The composite model upper-bound model consists of two scenarios. When M_a is greater than or equal to R , the execution efficiency value depends on the value of aggregate CPU loading. In the second scenario, when M_a is less than R , the execution efficiency value depends on the composite effect of CPU and memory.

The composite CPU and memory availability lower bound model, denoted by $\underline{cm}$, represents the worst case

efficiency value of the machine for thread execution is defined by the product of CPU and memory lower bound.

$$\underline{cm} = \begin{cases} \frac{1}{1 + \left(\frac{n-1}{n} \right) \times \left(\frac{L}{r} \right) - \mu} \times \frac{\tau}{\tau + A_p \times \sum_{i=1}^n R_i}, & \text{if } M_a \geq R, \\ \frac{1}{1 + \left(\frac{n-1}{n} \right) \times \left(\frac{L}{r} \right) - \mu} \times \frac{\tau}{\tau + (A_p \times M_a) + (A_v + A_p + A_c)} \times \left(\left(\sum_{i=1}^n R_i \right) - M_a \right) + \eta & \text{else.} \end{cases} \quad (18)$$

A new parameter, μ , is incorporated to represent the probability of worst-case staggering of work-phases. When the value of L is less than or equal to r , the value of μ is $L/(n \times \zeta)$ because of the small probability of worst-case staggering. When the value of L is greater than r , the value of μ becomes $L/(n + r \times \zeta)$ because of increased work-phase overlap probability.

The lower-bound of the composite prediction model consists of two scenarios. When the value of M_a is greater than or equal to R , the execution efficiency value is derived from the product of CPU availability lower-bound and memory models' first lower-bound expression. In the second scenario, when M_a is less than R , the execution efficiency value is derived from the product of CPU availability lower-bound model and the memory lower-bound models' second expression to reflect the composite effect accurately.

Figure 12 (a) Upper- and lower-bound surfaces of the composite prediction model associated with equations (17) and (18), respectively for multi-core systems with $r = 4$ and $n = 16$ (b) Surface diagram of the composite upper-bound model for $r = 4$ and $\tau = 1$ sec (c) Surface diagram of the composite upper-bound model for $r = 4$ and $\tau = 100$ sec (see online version for colours)

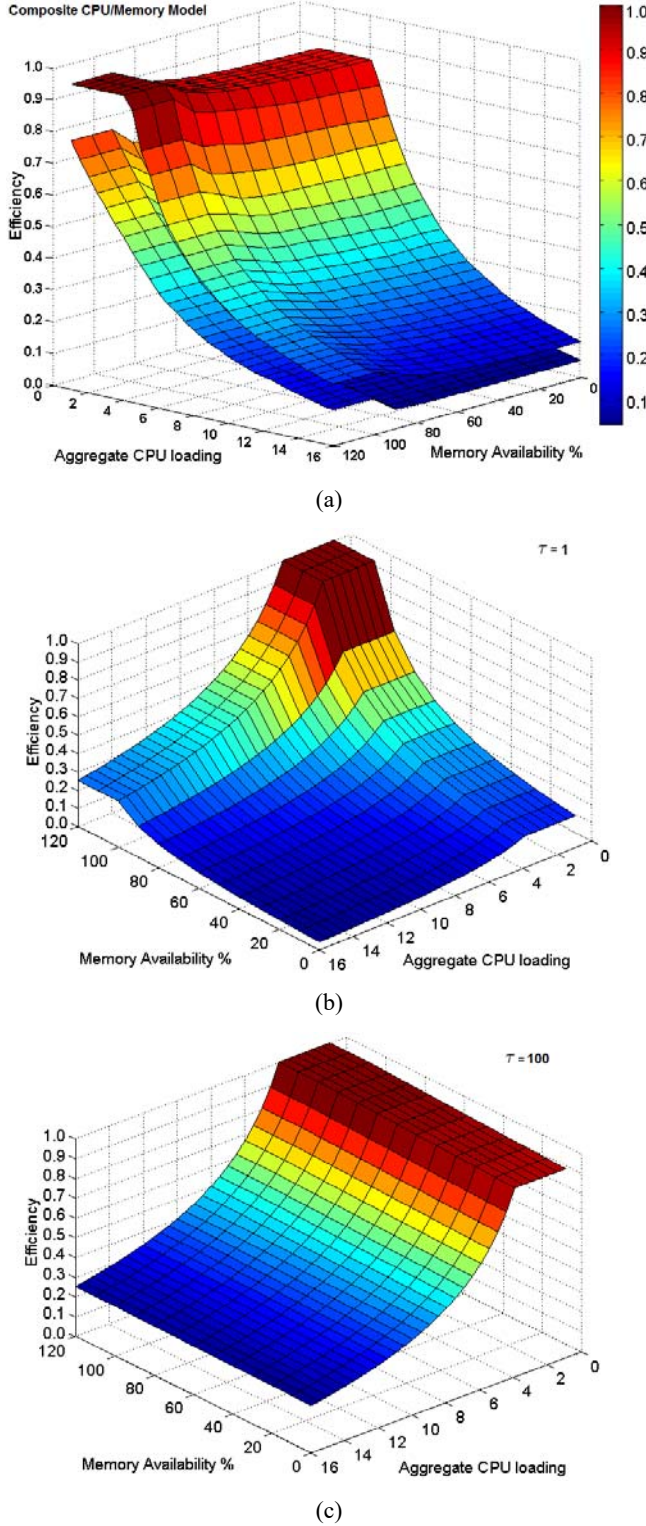


Figure 12(a) shows surfaces of the theoretically derived upper- and lower-bound model in which horizontal axes represent aggregate CPU loading (of the set of tasks) and

memory availability percentage (with respect to memory requirements of the set of tasks), and the vertical axis represents the overall efficiency of the machine. It can be observed that for CPU loading values greater than the number of cores (here, $r = 4$), the idealised function for $\bar{e}$ decreases according to the ratio of the number of cores to the total CPU loading. The significant degradation of efficiency value can also be observed when the memory availability for threads decreases accordingly from 120% down to 0%. Ideally, if a machine's CPU and memory resources are both lightly loaded, then the efficiency of the machine is at or near its maximum value.

Figures 12(b) and (c) show the effect of changing the value of τ in the upper-bound model. The purpose of this figure is to show the effect of ideal thread execution time (τ) when the value changes from 1 to 100 sec. It can be observed among surfaces that when the value of τ increases from 1 to 100, the overall efficiency of the node increases considering the memory requirement is same in all three cases. The efficiency value increases because when the program runs for longer period of time, the page fault overhead (which is same for all the three cases) has less impact compared to a program which runs for a shorter period of time. A similar effect is illustrated in Figure 8 of Section 4.1.

In the following section, experimental studies are performed to measure processor efficiency while executing threads to verify that the introduced upper- and lower-bound composite models can estimate the actual empirical measured efficiency surface.

5.2 Experimental studies

5.2.1 Overview

The benchmark programs used for measuring the overall efficiency of a machine is presented in Table 1. The *Bitonic Sorting Network*, *Sparse matrix vector multiplication*, and *Tridiagonal Solver (using Gaussian elimination)* program consists of expressions that are mix of CPU and memory related operations and thus ideal for the composite model empirical studies. Similar approaches are taken for modelling threads by a series of alternating work and sleep phases as described in Section 3.2.1. The purpose of the experimental study is to empirically measure the efficiency of the machine as a function of aggregate CPU loading and memory availability factors. Aggregate CPU loading and memory requirement values are selected randomly and distributed among threads by using Algorithms 1, 2, and 3, respectively. Uniform sampling of data across the values of possible aggregate CPU loading and memory availability has been carried-out. For implementing the benchmark and framework programs, most of the modules of the CPU and memory availability programs are reused.

About 18,000 test runs are conducted for three independent sets of test cases in which eight, 12, and 16 threads are spawned concurrently in a quad-core machine. This vast number of test runs is conducted to sufficiently cover the possible scenarios of thread executions. A similar

experimental environment as described in Section 3.2.2 has also been deployed here for conducting the case studies.

5.2.2 Benchmarking

A similar but combined approach has been performed (described in Sections 3.2.3 and 4.2.3) for setting up the empirical framework programs of the composite model. For the memory portion, each thread was assigned an array size of 50MB and the data is generated randomly for initialise the array. For the CPU portion, sleep phase length and total work load values are calculated by utilising equations (8) and (9), respectively. The thread with the minimum CPU load was assigned 1.0×10^8 units of work load, which should be accomplished in 25 work-sleep phases. In the work component of each phase, threads accomplish 4.0×10^6 units of fixed computational work. Thus, when the memory requirements of threads is less than the available memory of the system, equation (19) can be used to determine the ideal thread execution time, denoted by v . Input parameters for computing ideal execution time (v) are work phase time in the quantum (Q_w), number of work phases to complete the total work (N_w), and CPU usage factor of the thread (L_i).

$$v = \left((Q_w \times N_w) + (N_w - 1.0) \times \left(\frac{Q_w \times (1.2 - L_i)}{L_i} \right) \right) \quad (19)$$

According to equation (19), the value of v can be measured dynamically using the work phase length (measured by multiple test runs), number of work phases (total work/work accomplished in a single phase), and the CPU usage factor of the thread. All parameter values of equation (19) are available while the program is running. As the total amount of work depends on the CPU load of threads, the variable ideal thread execution time for threads (because of variable work load) can be measured accurately by utilising the equation (19).

5.2.3 Case studies and results

For measuring the overall efficiency value of thread execution, three independent case studies were conducted on a quad-core machine for eight, 12, and 16 threads. Aggregate CPU load and memory availability values were selected randomly and distributed among the threads as described in Sections 3.2.1 and 4.2.1, respectively. As the focus of this empirical study is to measure the effect in overall efficiency of machines when both CPU and memory usage are varied, the number of threads deployed is always above the number of CPU cores of the machine and the memory availability is varied from 120% down to 0%.

Figure 13 (a) Measured efficiency surface of eight threads in a quad core machine for $\tau = 30$ and number of test runs are 4,000 (b) The measured efficiency surface is superimposed with composite upper- and lower-bound surfaces (c) Same surfaces in an alternative perspective for clear visualisation of the binding of the efficiency surface by upper- and lower-model surfaces (see online version for colours)

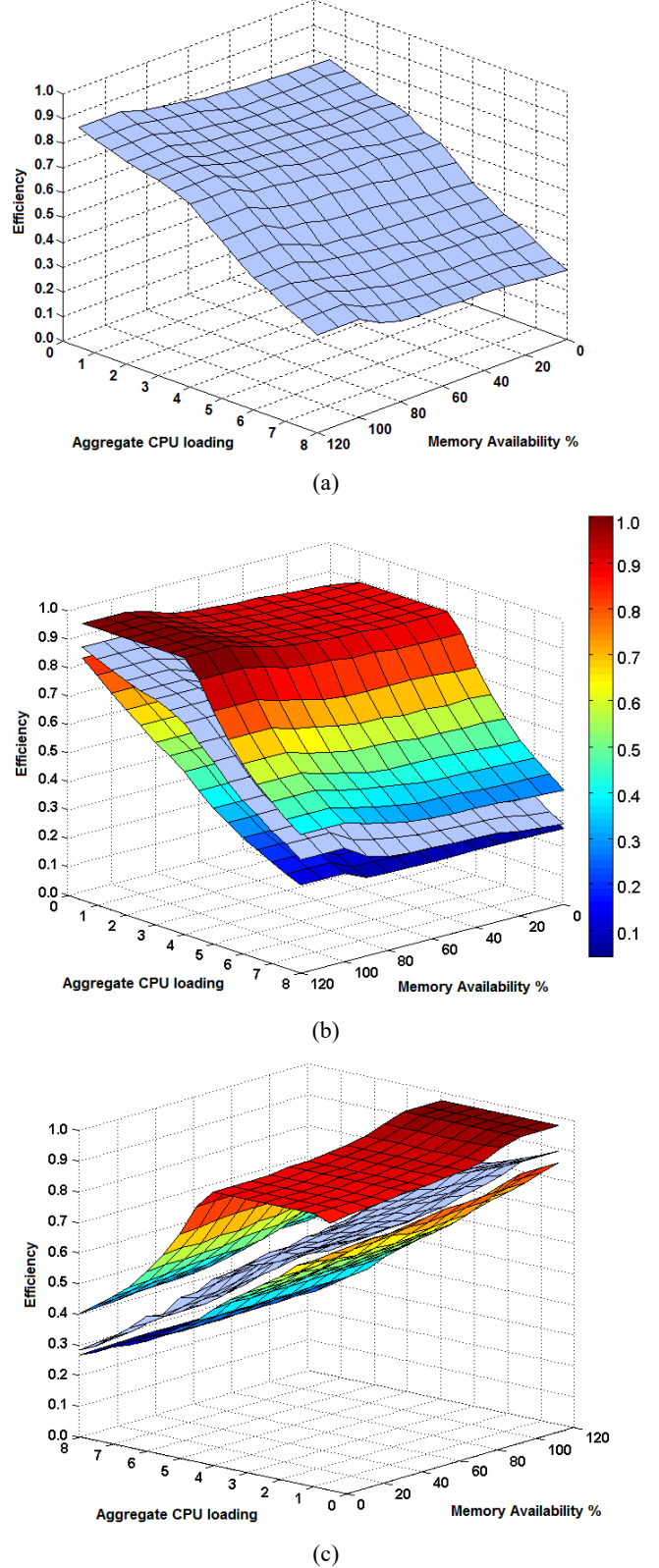


Figure 14 (a) Measured efficiency surface of 16 threads in a quad-core machine for $\tau = 30$ and number of test runs are 8,000 (b) The measured efficiency surface is superimposed with composite upper- and lower-bound surfaces (c) Same surfaces in an alternative perspective for clear visualisation of upper- and lower-limit binding of the efficiency surface (see online version for colours)

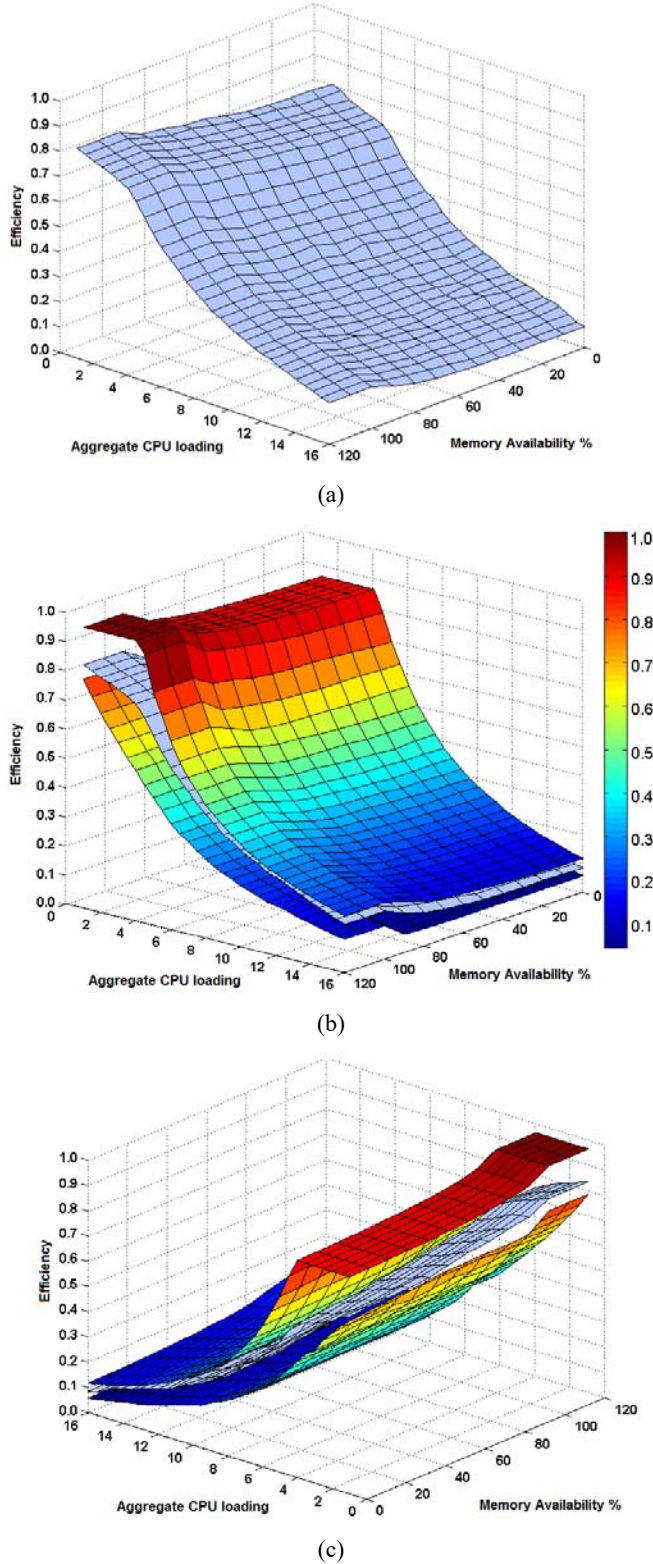


Figure 13(a) shows measured efficiency surface for the execution of eight threads in a quad-core machine. About 4,000 independent test cases are carried out to capture all possible execution efficiency scenarios due to CPU and memory availability variation. A moving average is taken from these test results with a sliding window size of 0.10 aggregate CPU loading and 0.5% of memory availability, and incremental value of 0.01 CPU loading and 0.5% memory availability. The data is then converted to a two dimensional matrix format for plotting a 3D surface diagram.

It can be observed from Figure 13(a) that the efficiency value decreases significantly when the aggregate CPU load reaches beyond the CPU cores (here, $r = 4$) because of the increased CPU contention among running threads. Figure 13(a) also illustrates the decrease of machine efficiency because of the decrease of memory availability. The efficiency decrease due to memory availability is moderate because the τ value of test runs are large (refer to Figure 8; here, $\tau = 30$ sec).

In Figures 13(b) and 13(c), the efficiency surface is superimposed with upper- and lower-bound surfaces associated with equations (17) and (18), respectively. The purpose of Figures 13(b) and 13(c) is to verify that the efficiency surface shown in Figure 13(a) is bound using the composite upper- and lower-bound models. From the empirical results and measured efficiency surface plot in Figure 13(b), it is apparent that theoretically derived upper- and lower-limits introduced in this section do bound the actual measured efficiency surface very well (note: the efficiency surface diagram is given a different and light colour so that any overlap can be detected immediately). It can also be seen from Figure 13(b) that bounds are tighter when the aggregate CPU and memory loading is either relatively low or relatively high. Figure 13(c) shows the same surface diagrams in a different perspective to illustrate that there are no crossings among measured efficiency, upper-, and lower-bound surfaces. This figure also illustrates the degradation in efficiency when the memory availability decreases.

In the next set of test cases, 16 threads were spawned concurrently for measuring the overall machine efficiency of thread execution. A similar approach has been taken, like eight threads, for conducting the empirical case studies. About 8,000 independent test cases were conducted in which CPU and memory availability were selected randomly and distributed among threads to cover all possible scenarios. Moving averages are taken from these empirical studies resulting data with a similar sliding window approach for generating efficiency surface data. Upper- and lower-bound surface data are generated from the same model equations (17) and (18), respectively.

Figure 14(a) shows the average measured efficiency surface for 16 threads in a batch. A similar decrease in efficiency has also been observed when CPU loading increases and memory availability decreases. Figures 14(b) and 14(c) illustrate the efficiency surface superimposed with theoretically derived upper- and

lower-bound surfaces. From Figures 14(b) and 14(c), it is apparent that the upper- and lower-bound prediction models introduced here can also bound the actual measured efficiency surface very well.

Experimental studies and statistical analysis of eight, 12, and 16 concurrent threads conducted independently validates the accuracy of the introduced prediction models. The proposed composite prediction model provides a basis of an empirical model for estimating execution efficiency of processes while CPU and memory resources are uncertain.

6 Conclusions

This paper has developed analytical models (and conducted empirical studies) for predicting (and measuring) efficiency of multi-core machines for a set of tasks. It is necessary to have accurate models for estimating CPU and memory resources because of the dynamic nature of computer systems and their workload. The composite prediction model proposed in this paper is derived from the CPU availability and memory models. The CPU availability and memory models have been introduced for predicting the overall machines' efficiency for concurrent thread execution on a time-shared system. The prediction models were validated empirically by an extensive set of case studies involving real-world benchmark programs having dynamic behaviours. Additionally, an average CPU availability prediction model is introduced and validated empirically. The introduced average CPU availability model is suitable for multi-core systems with hyper-threading enabled.

The results of empirical studies are presented in this paper in the form of scatter plots, surfaces, and tables. Scatter plots provide a clear visualisation of measured efficiency based on the density of the dots in the plot for CPU availability and memory models. For the composite prediction model, surface plots for measured thread execution efficiency along with upper- and lower-bound surfaces provide a clear visualisation of all possible cases of combined CPU and memory availability variation in thread execution. These empirically measured availability values are shown to generally fall within theoretically derived upper- and lower-bound models.

The introduced composite prediction model can be used as a building block for a distributed task scheduler to determine the order (or find sub sets) in which tasks should be assigned to compute nodes for minimising the total execution time prior to its placement in the run queue. All the obtained results justify the strength of the introduced models for predicting the efficiency of a machine while executing threads. Hence, the ability of these models to predict the CPU availability and processor efficiency for thread execution while the resources (CPU and memory) availability is uncertain in dynamic environment has been demonstrated. Thus, the usefulness of the introduced prediction models in real-world applications for estimating the execution efficiency of tasks before they are placed into the run-queue by a scheduler has been motivated, and is the topic of future studies.

Acknowledgements

We would like to thank Jonathan Mullen, System Administrator, School of Computer Science, University of Oklahoma, for his time and coordinated support. We would also like to extend our gratitude to Dr. Mahendran Veeramani for his valuable feedback towards this work.

References

- Aioanei, R. (2011) *Linux Process Scheduling Made Easier*, News in LinuxCareer.com [online] <http://how-to.linuxcareer.com/linux-process-scheduling-made-easier-lance> (accessed July 2012).
- Avi, S., Peter, G.B. and Greg, G. (2010) *Operating System Concepts*, 8th ed., 26 January, John Wiley and Sons, USA, ISBN: ISBN:978-0-470-50949-4, 26 January.
- Beltrán, M. and Guzmán, A. (2009) 'How to balance the load on heterogeneous clusters', *International Journal of High Performance Computing Applications*, Vol. 23, No. 1, pp.99–118.
- Beltrán, M., Guzmán, A. and Bosque, J.L. (2008) 'A new CPU availability prediction model for time-shared systems', *IEEE Transactions on Computers*, Vol. 57, No. 7, pp.865–875.
- Bryant, R.E. and O'Hallaron, D.R. (2011) *Computer Systems, A Programmer's Perspective*, 2nd ed., Prentice Hall Inc., NY, USA, ISBN: 978-0130340740.
- Carlsson, N., Williamson, C., Hirt, A. and Jacobson Jr., M. (2012) 'Performance modeling of anonymity protocols', *Journal of Performance Evaluation*, Vol. 69, No. 12, pp.643–661, Elsevier.
- Cepeda, S. (2012) *Intel Hyper-Threading Technology: Your Questions Answered* [online] <http://software.intel.com/en-us/articles/intel-hyper-threading-technology-your-questions-answered> (accessed 22 February 2013).
- Chen, Y-C., Wang, S-T., Chang, H-Y., Chen, T-M. and Li, C-H. (2011) 'The performance analysis for virtualisation cluster and cloud platforms', *Int. J. of Computational Science and Engineering (IJCSE)*, Vol. 6, No. 4, pp.255–263, DOI: 10.1504/IJCSE.2011.043924.
- Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2009) *Introduction to Algorithms*, 4th ed., Prentice-Hall, USA, ISBN-10: 0262033844.
- Dinda, P.A. (2001) 'Online prediction of the running time of tasks', *Proc. 10th IEEE International Symposium High Performance Distributed Computing*, San Francisco, CA, pp.383–394, ISSN: 1082-8907.
- Dwyer, T., Fedorova, A., Blagodurov, S., Roth, M., Gaud, F. and Pei, J. (2012) 'A practical method for estimating performance degradation on multicore processors, and its application to HPC workloads', *SC '12 Proc. of the Int. Conference on High Performance Computing, Networking, Storage and Analysis*, Article No. 83, ISBN: 978-1-4673-0804-5, IEEE Computer Society Press Los Alamitos, CA.
- Fedorova, A., Vengerov, D. and Doucette, D. (2007) *Operating System Scheduling On Heterogeneous Core Systems*, Sun Microsystems Technical Report [online] <http://www.techrepublic.com/whitepapers/operating-system-scheduling-on-heterogeneous-core-systems/314436> (accessed 3 May 2014).

- Garay, G.R., Ortega, J., Díaz, A.F., Corrales, L. and Alarcón-Aquino, V. (2013) 'System performance evaluation by combining RTC and VHDL simulation: a case study on NICs original research article', *Journal of Systems Architecture*, Part D, Vol. 59, No. 10, pp.1277–1298, Elsevier.
- Grounds, N.G., Antonio, J.K. and Muehring, J. (2009) 'Cost-minimizing scheduling of workflows on a cloud of memory managed multicore machines', *CloudCom 2009, LNCS*, Vol. 5931, pp.435–450.
- Hasan, K.S. (2011) 'A distributed chess playing software system model using dynamic CPU availability prediction', *International Conference on Software Engineering Research and Practice (SERP-11)*, Sponsor: World Academy of Science and Computer Science Research, Education, and Applications (CSREA), Las Vegas, Nevada.
- Hasan, K.S., Antonio, J.K. and Radhakrishnan, S. (2014) 'A new composite CPU/memory model for predicting efficiency of multi-core processing', *The 20th IEEE International Symposium on High Performance Computer Architecture (HPCA-2014) workshop*, Sponsored by: IEEE Computer Society, Orlando, FL.
- Hasan, K.S., Grounds, N.G. and Antonio, J.K. (2011) 'Predicting CPU availability of a multi-core processor executing concurrent Java threads', *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA 11)*, Sponsor: World Academy of Science and Computer Science Research, Education, and Applications (CSREA), Las Vegas, NV.
- Hasan, K.S., Radhakrishnan, S. and Antonio, J.K. (2013) 'Composite prediction model and task distribution on a cloud of multi-core processors', *The IEEE Conference on High Performance Computing (HiPC-2013) Workshop on Cloud Computing Applications (IWCA-13)*, Bangalore, India.
- Hu, J., He, Y., Zhuge, Q., Sha, E.H.-M., Xue, C.J. and Zhao, Y. (2013) 'Minimizing accumulative memory load cost on multi-core DSPs with multi-level memory original research article', *Journal of Systems Architecture*, Vol. 59, No. 7, pp.389–99, Elsevier.
- Jones, M.T. (2006) 'Inside the Linux scheduler 2.6', *The Journal of a Linux Sysadmin* [online] <http://www.ducea.com/2006/07/08/inside-the-linux-scheduler/>.
- Kumar, A. (2008) *Multiprocessing with the Completely Fair Scheduler for Linux*, IBM Technical Report, <http://www.ibm.com/developerworks/linux/library/l-cfs/author1>.
- Lee, W.Y. (2013) 'Energy-efficient scheduling of periodic real-time tasks on lightly loaded multicore processors', *IEEE Transactions on Parallel and Distributed Systems*, Vol. 23, No. 3, pp.530–537, ISSN: 1045-9219.
- Paessler Knowledge Base (2011) *Monitoring Running Processes in Linux* [online] <http://www.paessler.com/knowledge-base/en/topic/29403-monitoring-processes-in-linux> (accessed 15 July 2013).
- Shrestha, H.K., Grounds, N., Madden, J., Martin, M., Antonio, J.K., Sachs, J., Zuech, J. and Sanchez, C. (2009) 'Scheduling workflows on a cluster of memory managed multicore machines', *Proceedings of the International Conference on Parallel and Distributed Processing Techniques and Applications (PDPTA 09)*, Las Vegas, NV.
- Sotiriadis, S., Bessis, N., Antonopoulos, N. and Hill, R. (2013) 'Meta-scheduling algorithms for managing inter-cloud interoperability', *Int. J. of High Performance Computing and Networking (IJHPCN)*, Vol. 7, No. 3, pp.156–172, DOI: 10.1504/IJHPCN.2013.056518.
- Tanaka, B.K. (2005) 'Monitoring virtual memory with vmstat', *Linux Journal*, 31 October 2005, No. 140, Article: 8535, <http://www.linuxjournal.com/article/8178>.
- Tanenbaum, A.S. (2008) *Modern Operating Systems*, 3rd ed., 21 December 2007; ISBN-13: 978-0136006633; Upper Saddle River, NJ, 07458, USA.
- Vaidya, V.G., Sah, S. and Ranadive, P. (2010) 'Optimal task scheduler for multi-core processor', *2nd International Conference on Software Technology and Engineering (ICSTE)*, Center for Research in Engineering Sciences and Technology, KPIT Cummins Infosystems Ltd., Pune, Maharashtra, India.
- Wolski, R., Spring, N. and Hayes, J. (2002) 'Predicting the CPU availability of time-shared UNIX systems on the computational grid', *Proc. 8th International Symposium on High Performance Distributed Computing*, pp. 105–111, ISBN: 0-7803-5681-0.
- Yeh, J.-C., Lin, C.-H. and Liu, C.-N. (2014) 'Multi-core system performance prediction and analysis at the ESL', *Int. J. of Computational Science and Engineering*, Vol. 9, Nos. 1/2, pp.86–94, DOI: 10.1504/IJCSE.2014.058700.
- Yuen, C.-H.P. and Chan, S.H.G. (2012) 'Scalable real-time monitoring for distributed applications', *IEEE Transactions on Parallel and Distributed Systems*, Vol. 23, No. 12, pp.2330–2337, ISSN: 1045-9219.
- Zhang, J., Sha, E.H.-M., Zhuge, Q., Yi, J. and Wu, K. (2014) 'Efficient fault-tolerant scheduling on multiprocessor systems via replication and deallocation', *Int. J. of Embedded Systems (IJES)*, Vol. 6, Nos. 2/3, pp.216–224, DOI: 10.1504/IJES.2014.063819.
- Zhang, Y., Sun, W. and Inoguchi, Y. (2006) 'Predicting running time of grid tasks on CPU load predictions', *Proc. 7th IEEE/ACM International Conference on Grid Computing*, Barcelona, pp.286–292, ISBN: 1-4244-0343-X.